

Department of Electrical Engineering



From Dynamic Spectra to Burst Catalogs: An AI-Driven Framework for Automated Identification and Classification of Solar Radio Bursts

by

Ade AP

MSc Thesis

Thesis committee

Prof.dr. Ulf Johannsen (chair)
Prof.dr. M.J. Bentum
Prof.dr. Roel Jordans
Dr. J. Lazendic-Galloway
Dr. P. Zucca
Dr. M. Mancini

Graduation

Program: Masters Artificial Intelligence and Engineering Systems
Capacity group: Electromagnetics - Radio Science Lab
Thesis supervisors:
Prof.dr.ir. M.J. Bentum
Dr. J. Lazendic-Galloway
Dr. P. Zucca
Collaborators:
Dr. M. Mancini
Date of defense: 7 January 2026
Student ID: 1945580
Study load (ECTS): 45

The research of this thesis has been carried out in collaboration with ASTRON, the Netherlands Institute for Radio Astronomy. This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

From Dynamic Spectra to Burst Catalogs: An AI-Driven Framework for Automated Identification and Classification of Solar Radio Bursts

A.P. Ade - a.p.ade@student.tue.nl

Abstract—This study develops an AI-driven framework for automated identification and classification of solar radio bursts from LOFAR telescope. An unified data pipeline is introduced to construct a consistent machine-learning dataset, incorporating orientation correction, interference mitigation, and systematic label generation techniques. Solar burst recognition is formulated as a multi-label classification problem to reflect realistic 15-minute observation windows. To address label scarcity and class imbalance, a model is constructed to generate synthetic data using a conditional deep convolutional generative adversarial network. Furthermore, a novel self-supervised machine learning model is developed to exploit large volumes of unlabeled data and learn invariant radio-morphological representations in pre-training phase prior to supervised fine-tuning. Experimental results show that self-supervised models outperform purely supervised baselines across multiple evaluation metrics, demonstrating improved robustness and reduced reliance on manual labeling. The proposed framework enables scalable and deployable SRB cataloging of LOFAR solar burst observations.

I. INTRODUCTION

Solar Radio Bursts (SRB) are short-to-long episodes of enhanced radio emission produced when solar activity (such as flares, magnetic reconnection, and coronal mass ejections (CMEs)) accelerates electrons and disturbs the coronal plasma. Those electrons and plasma disturbances generate radio emission that can be traced as bright structures in a “dynamic spectra” images (intensity as a function of observing frequency and time) captured by a radio telescope. Because the emission mechanism depends on plasma conditions and how electrons propagate, different physical scenarios produce different “signatures” in the time–frequency plane of a dynamic spectrum observation [?].

While solar radio bursts are widely used as diagnostics for energetic particle acceleration and shock/eruption physics, they also matter operationally because certain burst types correlate with space-weather hazards and can act as early warning signatures for geo-effective activity [?], [?], [?], [?], [?]. Therefore, a prompt identification of the presence and types of bursts in a large stream of raw observations is needed, which can be turned into an categorized catalog that enables both research and monitoring.

Low Frequency ARray (LOFAR) is a radio telescope, operated by ASTRON (the Netherlands Institute for Radio Astronomy) and designed to observe 10–240 MHz band, which overlaps well with the frequency ranges where many coronal radio burst signatures are visible. It observes solar activity daily (continuously or in intervals), thus producing a

massive archive of data, which is automatically processed into 15-minute long dynamic spectra images, as shown in figure 1.

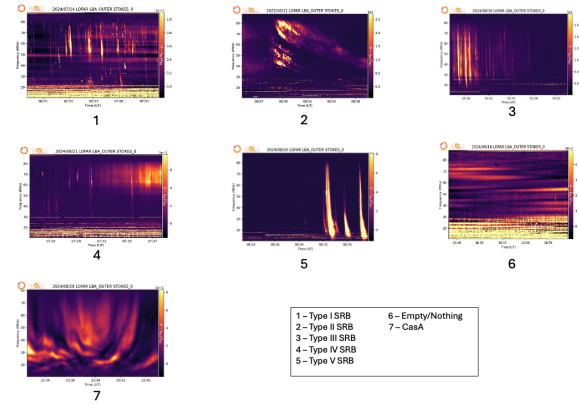


Fig. 1: Representative dynamic spectra from LOFAR observations, showing all considered classes, including Type I–V solar radio bursts, empty class (6) where observations are dominated by background noise, and class 7 which is Cassiopeia A (Cas A) calibration data (see Section VIII-C2 for more detail).

However, these large volumes of dynamic spectra need manual inspection to establish the presence and nature of solar bursts (labeling), making the categorization process extremely slow. Furthermore, classifying SRBs into appropriate categories (Types I to V) is also important because each type represents different underlying plasma processes, which is critical for understanding the associated physical phenomena [?]. This study addresses this bottleneck through the following research questions: (RQ1) Can physically meaningful SRB morphology be reliably recovered from realistic LOFAR dynamic spectra using image-based machine learning? (RQ2) Can unlabeled radio observations be exploited to learn invariant representations of burst physics and reduce the reliance on manual labels?

Section II reviews related work on SRB classification and highlights their limitations when applied to realistic LOFAR dynamic spectra. Section III describes the data and pre-processing pipeline. Section IV establishes a supervised multi-label baseline and introduces the generative data augmentation technique. Section V presents a self-supervised learning framework and a fine-tuning strategy. Section VI presents a discussion of results, followed by conclusions and future

work in Section VII. Detailed network architectures, training procedures, and implementation-specific parameters are provided in the appendices to maintain clarity in the main text. The full code implementation and documentation is available at <https://github.com/AddyAde10/LOFAR-AI-detection-System-for-Astron.git>.

II. EXISTING METHODS

From a machine-learning perspective, simple classification techniques are not effective enough to differentiate SRB types. The same physical phenomenon can appear with different drift rates (how quickly the observed emission frequency changes with time), and the background emission can also be structured and time-varying. Different burst families can overlap or even resemble each other morphologically (e.g., Type III and Type V, see Fig 1), which makes the decision boundary to correctly classify SRB ambiguous even before adding instrumental effects such as noise [?], [?]. This sets a high bar: a classifier must be robust to learn general morphological patterns of the bursts while being invariant to changing backgrounds, Radio Frequency Interference (RFI), and mixed/overlapping events.

Early automated pipelines for dynamic spectra leaned heavily on deterministic signal processing and image-processing heuristics—background estimation, thresholding, morphology operators, and rule-based post-processing to extract candidate burst pixels or connected components before declaring detections [?], [?]. These approaches are attractive because they are interpretable and can run fast, but they tend to be sensitive to the exact noise statistics and interference patterns of the instrument. Consequently, such methods struggle in regimes critical for LOFAR archives, which produce high-volume data streams (up to 3 Gb/s) characterized by heavy RFI [?], [?]. In these environments, fixed heuristics often either misses faint events (low recall) or triggers aggressively on interference (low precision), highlighting a persistent gap in robust, transferable pre-processing and evaluation [?].

Deep learning methods moved the field from rule-based features toward learned spectrogram image representations using Neural Networks. For example, Guo et al. [?] combines convolutional feature extraction with sequence modeling (CNN + recurrent units) and apply pre-processing steps such as normalization and segmentation-style operations to make burst structures more salient in the input, demonstrating that learned models can recognize burst patterns more flexibly than pure heuristics [?]. In parallel, transfer-learning studies fine-tuned a pre-trained image classifier like ResNet50 on burst images and reported strong accuracies on curated benchmarks, showing that off-the-shelf vision models can be repurposed for radio-spectrogram morphology. The recurring drawback in similar deep learning research is that many of their pipelines implicitly simplify the problem to “one image, one label” (or even just a binary Type II vs Type III task), without addressing multi-burst composites, weak labels, or temporal/frequency localization; they also often rely on tightly controlled datasets where testing metrics are insufficient for deployment conclusions. This leaves an important research gap for LOFAR-

scale deployment: learned classifiers must be validated under realistic archive conditions (multi-label and/or weak labels) and tighter metrics suitable for this task specifically [?], [?], [?].

Object detection reframes SRB analysis by predicting where in the time–frequency plane a burst occurs using bounding boxes (BB) classification, enabling localized burst information and potentially more actionable catalogues [?], [?]. Scully et al. demonstrate this direction for Type III bursts by combining a YOLO (You Only Look Once) detector with synthetic augmentation, reporting that detection can be automated at scale and that generated examples can reduce manual labeling load [?], [?]. The limitation, especially for LOFAR dynamic spectra, is representational. Bounding Boxes are a coarse fit for SRB morphology, so “correct” boxes can be ambiguous and detectors can under-perform in crowded scenes where multiple burst types co-exist in the same frame. In practice, BB pipelines also introduce a heavy annotation burden (making boxes and labeling them). Additionally, the reported performance can be difficult to interpret if testing is not done on sufficiently diverse days and interference conditions. These issues point directly to gaps emphasized by this research: (i) localization should not be constrained to boxes when the underlying structure is non-rectangular and multi-instance, and (ii) evaluation must be designed to reflect the data distribution (eg, class imbalance), otherwise high scores may not translate to reliable categorized archive.

Zhang et al. [?] propose a conditional deep convolutional GAN (cDCGAN) to mitigate class imbalance and report strong recognition accuracies on Culgoora/Learmonth dynamic spectra later compared with this study in section [VI]. The generative angle is a genuinely useful contribution because conditional synthesis is a reasonable strategy for expanding rare SRB classes when real observations are scarce [?], [?], [?]. However, their classification protocol simplifies the SRB problem in ways that reduce realism for operational use. First, the pipeline is presented as a *single-label* classifier [?]. This is a serious limitation because real dynamic spectra frequently contain multiple overlapping or sequential phenomena, so a single-label assumption can inflate apparent performance relative to realistic multi-burst conditions. Second, their evaluation protocol is not sufficiently explicit about *real-only* testing. The paper states that the classifier takes as input “both the samples generated by G and the real samples” [?]. Synthetic data generation is kept in training-augmentation loop, but if generated samples are included in evaluation, then reported test accuracy can become optimistic due to generated samples contaminating the test dataset. Conceptually, that is like a student (model) writing some of the exam questions themselves before sitting the exam (seeing **unlabeled** synthetic data). The score may look impressive, but it is not a reliable measure of generalization to unseen real observations. This concern matches the standard ML evaluation guidance on preventing train–test contamination and other leakage pathways that yield misleading performance estimates [?], [?].

Finally, there is a clear opportunity around efficient usage

of data and labels. Solar Radio Bursts archives are dominated by unlabeled or weakly labeled observations, yet they contain rich amount of meaningful data that is useful for learning general spectrogram representations. Work in adjacent radio-astronomy settings shows that a Representation Learning technique called Self-Supervised Learning (SSL) can learn useful features from large unlabeled corpora and improve classification under limited labels [?], [?], [?]. However, SSL is not yet a default baseline in SRB classification pipelines, even though the data regime (massive unlabeled archives and scarce high-quality labels) strongly favors it. Building on these gaps, this study focuses on what is required—data engineering, supervised and representation learning, and robust evaluation—to teach a neural network to classify multiple SRB types reliably on LOFAR dynamic spectra, including challenging cases such as heavy background structure, radio frequency interference (RFI), and class imbalance, while explicitly targeting label efficiency via self-supervised representation learning on unlabeled LOFAR spectrograms [?].

From an electromagnetic perspective, SRBs represent non-stationary plasma emission processes whose signatures are encoded as time–frequency structures shaped by electron acceleration, magnetic field topology and propagation effects. The central challenge is that these physical processes manifest as overlapping, drifting and broadband features whose morphology varies strongly with coronal conditions and instrumental response. This motivates data-driven approaches that can learn invariant morphological cues beyond hand-crafted heuristics.

III. DATA USED AND PROBLEM FORMULATION

A. Data used in this study

This study uses LOFAR solar dynamic spectra represented as two-dimensional time–frequency intensity images spanning 15-minute observation windows. Solar radio bursts are traditionally categorized into Types I–V based on their morphology in the time–frequency plane, which reflects the underlying emission mechanism and electron dynamics:

- Type I: short-lived, narrowband bursts often occurring in storms
- Type II: slowly drifting lanes associated with shock-driven emission
- Type III: rapidly drifting bursts produced by electron beams
- Type IV: broadband continuum emission
- Type V: short-duration continua following Type III activity

The complete procedure for retrieving and preparing the data is described in Appendix VIII. Solar radio bursts appear in these images as distinctive visual patterns, such as fast or slow drifting streaks, broadband continua, or clusters of short-lived features, as shown in Fig.[1]. Some intervals contain no bursts or are dominated by calibration observations. Therefore, this study considered seven data classes (labels), as shown in Fig 1:

- Solar Radio Bursts Type I–V (five morphological burst classes)

- Empty/quiet spectrum (no burst signature)
- Cas A calibration spectra

Each spectrum is standardized to a fixed resolution and stored together with metadata (timestamps, frequency range, and identifiers) in a structured the Hierarchical Data Format version 5 (HDF5) format, enabling efficient access and reproducible downstream analysis. The learning problem is therefore framed as recognizing characteristic burst morphologies in normalized spectrogram images.

A novel approach in this study is the first large-scale, fully automated labeling of previously unlabeled SRB data using National Oceanic and Atmospheric Administration (NOAA) Solar Event Reports. These reports are compiled daily by human analysts based on global solar monitoring data and list the occurrence of events using standardized descriptors, including start and end times, burst type (e.g., Type I–V), and frequency information. This study algorithmically aligns the reported event time intervals with the fixed 15-minute LOFAR spectra to assign burst labels in a reproducible manner, significantly expanding the available labeled dataset beyond the initial expert-annotated subset (from 2,854 labeled images to 6,555 labeled samples). Prior to model training, all spectra were corrected for a consistent time–frequency orientation and processed to reduce radio-frequency interference (RFI) and background fluctuations using robust background subtraction and median filtering. These preprocessing steps suppress instrumental and environmental artifacts while preserving genuine burst morphology. Full details of data acquisition, automated labeling logic, and RFI mitigation are provided in Appendix VIII.

B. Problem formulation

The dataset consists of 15-minute dynamic spectra, selected to reflect realistic operational observing conditions rather than curated, event-centered clips. This choice intentionally prioritizes physical realism and scalability over precise temporal localization, as operational solar monitoring systems must process continuous data streams.

From a physical perspective, each dynspec may contain multiple burst types or no bursts at all, reflecting the complex and overlapping nature of solar radio emission mechanisms. This motivates a multi-label formulation rather than single-event classification. Unlike single-label approaches, this avoids forcing an artificial hierarchy or exclusivity between burst types whose underlying physical processes are not mutually exclusive.

IV. DESIGNING SUPERVISED MACHINE LEARNING MODELS

A supervised model baseline represents a conventional machine-learning workflow in which a model is trained exclusively on labeled examples to predict burst types in previously unseen observations.

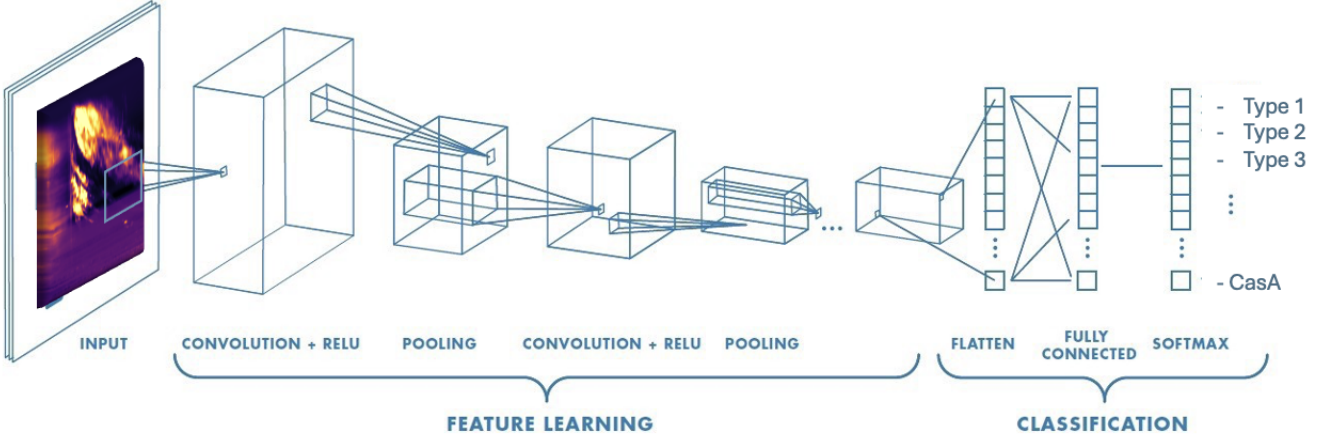


Fig. 2: Schematic overview of the Deep neural network architecture used for solar radio burst classification. The image is based on [https://wec.wur.nl/dse/42-ia2.html].

A. How Supervised Machine Learning works

In supervised machine learning, a model is trained using paired inputs (x_i, y_i) , where x_i is an input sample and y_i is its respective ground-truth label. In this study, each input x is a solar radio dynamic spectrum (dynspec), i.e., a time–frequency image whose intensity encodes received power and y are the respective labels to the spectra (e.g., $TypeI \rightarrow 1, TypeII \rightarrow 2, etc$). The task is to map these 2-D patterns to discrete burst categories to produce predicted labels $\hat{y}_i$. Dynamic spectra are widely treated as image-like representations for automated burst recognition and cataloging.

Deep Neural-Network classifiers are multiple layers of perceptrons (MLP) where every single neuron in one layer is connected to every single neuron in the adjacent layer, allowing for the transfer and integration of all information from the previous step to the next, forming a complete linkage essential for complex pattern recognition. This implements a parametric function $f_\theta(\cdot)$, parameterized by weights θ , that maps a new input image to a correct label/class prediction. In this setting, each dynspec is treated as an 800x500 matrix of frequency vs time readings, because burst signatures appear as structured patterns in the time–frequency plane. This is represented schematically in Fig. 2. Classification task transforms an input image into a set of unnormalized scores called logits $C = f_\theta(x)$, which are scores of “what the model thinks is in the image”. Convolutional neural networks (CNNs), which are also a part of Deep neural-networks, are commonly used for image inputs because their task is to learn hierarchies of spatial patterns by applying convolutional filters called kernels that detect local structures (such as edges, lines, and textures) and combine them through multiple layers to get images with increasingly abstract and task-relevant features. Early layers capture simple patterns, while deeper layers capture more abstract burst-specific morphology. The CNN output is a compact feature representation that is then converted into

class scores by a classification head section [?]. The forward pass produces logits in the output layer, which are converted into probabilities of a burst occurrence using equations [1 or 3]. These probability scores are then converted into binary decisions ($\hat{y}_i$) for a predicted burst class occurrence in the input image using a decision threshold which is mostly 0.5. The model then compares the predicted label $\hat{y}_i$ in the output layer with the ground truth label y_i to minimize a loss function like equation [4]. Model parameters (weights) θ are then updated via backpropagation by minimizing a loss function over mini-batches using gradient-based optimization. Figure 2 illustrates the end-to-end deep neural network pipeline adopted in this work for solar radio burst classification.

This study follows a standard Deep neural network model design called “You Only Look Once” (YOLO), where architecture consists of a backbone section for feature extraction, an optional neck section to aggregate features across different receptive-field scales, and a head section that produces final predictions [?], [?]. YOLO is primarily used for object-detection machine learning where the object of interest is localized in a bounding Box with a predicted label. In this study, the problem statement for SRB identification is used without any bounding boxes, the architecture is used purely as an efficient convolutional feature extractor for global burst presence classification. The objective of the supervised models is not event localization or parameter estimation, but reliable detection of burst presence based on global morphological signatures.

With classification-only use, the backbone produces feature maps, pooling layers summarize these feature maps by reducing spatial resolution, and the classification head (typically fully connected neural network layers) outputs a vector of class scores (logits). Pooling is used to (i) reduce computation, (ii) increase robustness to small shifts in the pattern location, and (iii) summarize feature evidence across the image. Training adjusts the weights θ to reduce a loss

function, which is a prediction error, that measures how different the model’s predictions are from the ground-truth labels. For single-label classification (exactly one class per image), logits z are converted to probabilities using softmax,

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}} \quad (1)$$

and optimized with cross-entropy loss,

$$\mathcal{L}_{CE}(x, y) = -\log(p_y) \quad (2)$$

where C is the number of classes, z_c is the logit for class C , and y is the true class index. For multi-label classification (multiple burst types can co-occur in one image), each class is treated independently using sigmoid probabilities,

$$p_c = \sigma(z_c) = \frac{1}{1 + e^{-z_c}} \quad (3)$$

with a binary cross-entropy (BCE) objective over a multi-hot target vector $y \in \{0, 1\}^C$,

$$\mathcal{L}_{BCE}(x, y) = -\sum_{c=1}^C [y_c \log(p_c) + (1 - y_c) \log(1 - p_c)] \quad (4)$$

These formulations correspond to the two supervision regimes later used in this study: an early single-label baseline (Model 1) and subsequent multi-label models (Model 2 and Model 3).

The loss decreases during training because the optimizer updates the weights in the direction that reduces the prediction error ($\hat{y}_i - y_i$). Concretely, at each iteration the model computes the loss for a mini-batch, backpropagation computes gradients $\nabla_{\theta} \mathcal{L}$ (how each weight contributes to the loss), and an optimizer performs an update such as

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (5)$$

where η is the learning rate. When training is progressing correctly, the predicted probabilities become more consistent with the true labels, which typically reduces training loss. The validation loss is monitored separately to check whether the improvements generalize beyond the training data.

Training proceeds by iterating over mini-batches: (i) forward pass to compute logits and probabilities, (ii) loss evaluation, (iii) gradient computation $\nabla_{\theta} \mathcal{L}$ via backpropagation, and (iv) parameter updates using an optimizer (e.g., SGD/Adam variants). [?]. Learning-rate schedules and regularization (e.g., weight decay, dropout) are commonly applied to stabilize convergence and reduce overfitting, particularly in small labeled datasets which used in the following section.

For SRB data, accuracy is generally not a sufficient metric, especially under class imbalance and multi-label co-occurrence. SRB corpora typically contain many quiet/“Nothing” spectra relative to rarer burst types, and multi-label spectra introduce partial correctness (e.g., detecting one burst type but missing a co-occurring type), so a model can achieve high accuracy by favoring common classes while failing on rare types [?]. For this reason, **“F1-based metrics”**

are emphasized: ‘Precision (P)’ measures how often predicted burst types are correct - important to avoid flooding catalogues with false alarms. ‘Recall (R)’ measures how often real bursts are found - important to avoid missing scientifically valuable events and biasing event lists toward only high-contrast detections [?]. **F1-score** balances these two, preventing the evaluation from being dominated by conservative predictions (high precision, low recall) or overly permissive predictions (high recall, low precision).

B. The Baseline Model (Model 1)

As an initial feasibility baseline, a supervised convolutional classifier was implemented on the filtered LOFAR dynamic-spectrum frames to establish an end-to-end training and evaluation pipeline. The baseline model focuses on a manually selected label set, of majority classes like (1,3,6) (Type I, Type III, and “no burst”), where multi-label samples were handled conservatively using a strict inclusion rule: only instances containing exactly one of the selected classes and no other burst labels were retained. The selection of these classes was based on the availability of enough labels for the supervised training process. Figure 9 shows the label distribution for selected classes for this model. Each frame was normalized per sample to [0,1], converted to a single-channel tensor, and resized to 224×224 image for training. The dataset was partitioned using stratified splits into train/validation/test subsets (70%,15%,15%), maintaining fair label proportion ratios across splits.

The network architecture followed a lightweight YOLO-style classification-only design: a stack of convolutional blocks with BatchNorm and SiLU activations, interleaved with C2F-style feature mixing modules and a spatial pyramid pooling fast (SPPF) block for multi-scale aggregation, followed by a small fully connected classification head with dropout feature. This choice was motivated by the effectiveness of YOLO-family convolutional backbones for learning discriminative time–frequency structure in burst imagery [?], where the backbone must capture sparse, elongated and drifting morphologies in dynspecs. The training used categorical cross-entropy loss with AdamW optimization and learning-rate reduction until the training loss curve plateaus.

Results for Model 1: Experiment 1, which trains exclusively on raw unfiltered observations, provides a strong but conservative baseline. The model achieves reliable performance on dominant and well-represented burst classes, as shown in Table I. However, performance for other classes could not be calculated, since data was too scarce for training and evaluation. These results motivates the next iteration of models to include all classes and multi-label prediction mechanisms.

Furthermore, SRB datasets are typically class-imbalanced, meaning rare burst types are provided with fewer labeled examples for supervised models to train on. Indeed, the retained subset still exhibits a strong skew toward dominant classes — a common issue in SRB classification. The dataset distribution used in this model is shown in Fig. 9 and Fig. 10. The prior

TABLE I: Model 1 (Baseline supervised classifier) evaluated on unfiltered (raw) test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.78	–	0.91	–	–	0.81	–
Recall	0.42	–	0.95	–	–	0.89	–
F1	0.54	–	0.93	–	–	0.84	–

work explicitly reports that improving balance by expanding under-represented categories is an effective lever for improving robustness and classification performance. Therefore, this study includes synthetic data generation, described in the next section.

C. Synthetic Data Generation using cDCGAN

To mitigate the limited availability of labeled dataset for training in several SRB categories and the resulting class imbalance, we generated additional training samples using a conditional Deep Convolutional Generative Adversarial Network (cDCGAN) [?]. The SRB literature notes that burst datasets are typically highly imbalanced and that GAN-based synthesis can be used to increase the effective sample size and reduce overfitting when minority classes are scarce [?]. Moreover, GAN-generated SRB images have been reported as sufficiently realistic to support the classifier and detection training, particularly for rare burst types where acquiring large corpora of real observations is difficult [?]. Architecturally, the design follows established DCGAN guidelines (convolutional up/down-sampling with batch normalization and nonlinearities, and a bounded output activation) that are known to improve training stability in convolutional GANs [?].

To begin the synthetic data generation, the model needs input data that is varied enough to produce even more varied synthetic bursts [?], [?]. The existing dynspec data for Type II, IV and V were augmented to around 1000 images per target class using dynspec-aware differentiable augmentations (DiffAug [?]). Augmentations like small random time/frequency shifts, cutouts (DiffAug-like masking) [?], contrast jitter, and noise injection improve robustness by preventing the discriminator from overfitting to superficial artifacts.

Training follows a standard adversarial procedure in which generator and discriminator are updated alternately until stable, label-consistent samples are obtained. Because adversarial losses do not converge monotonically, model selection is performed using a combination of visual inspection and relative distributional similarity across training checkpoints at a specified epoch, rather than absolute loss values. After training, synthetic dynamic spectra are exported in an HDF5 format fully compatible with the existing LOFAR preprocessing and dataloading pipeline and are used exclusively to augment the training set, ensuring that validation and test evaluations remain based on real observations.

Full architectural details, loss definitions, training procedures, and export specifications are provided in Appendix IX.

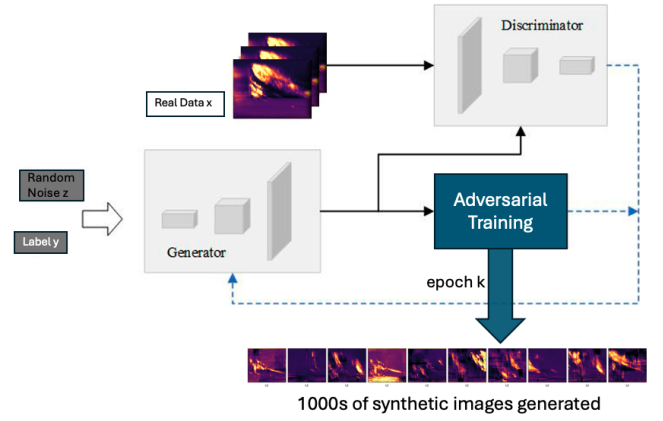


Fig. 3: Conditional DCGAN-based augmentation used in this study: following the cDCGAN strategy of Zhang *et al.* [?], we generated 1500 samples each for Type II at epoch 1500 (where an epoch denotes a training loop checkpoint), Type IV at epoch 1000, and Type V at epoch 500

D. Designing Multi-Label Supervised Classification Model (Model 2)

As established in the preceding baseline model section, supervised learning provides a natural first approach to automated SRB identification, but its performance was constrained by strong class imbalance, limited labeled data for rare burst types and single-label predictions. To address these limitations while preserving physical interpretability, the final supervised classifier is formulated as a multi-label image classification model.

As mentioned before, each LOFAR dynspec image may contain multiple burst types or no bursts at all as seen in Fig. 10. This requires multi-label classification. Instead of predicting one class ID, the model predicts a vector of independent probabilities, one per class, and then the model thresholds each probability to decide which bursts are present. This solution solves a major research gap of solving multiple burst presence in one SRB image and also avoids the need for defining bound boxes around each burst manually, a technique used in few other works [?], [?], [?]. Unlike object-detection approaches, this formulation avoids the need for manually defined bounding boxes and instead focuses on global burst morphology.

Following the data preparation and augmentation strategy described in the previous sections, synthetic samples generated using a conditional GAN are appended exclusively to the training set to improve representation of minority burst classes. Validation and test sets remain composed entirely of real LOFAR observations, ensuring that reported performance reflects genuine generalization rather than familiarity with generated data. This design addresses known shortcomings in earlier SRB studies where synthetic data were also used for evaluation [?].

The model itself is a YOLOv8-inspired model that learns

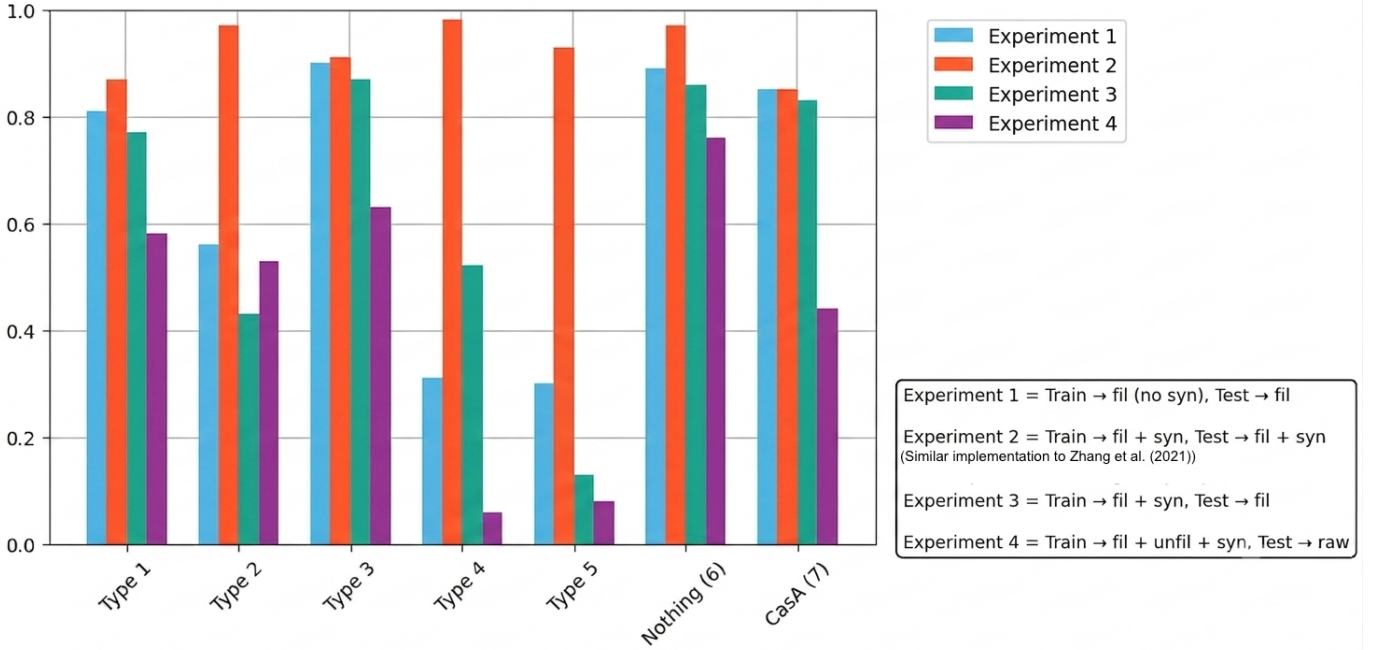


Fig. 4: Multi-label Supervised classifier (model 2) per-class F1-score performance comparison across four training/testing configurations (Experiments 1–4) for seven SRB classes, highlighting the impact of synthetic augmentation and domain shift (filtered vs. unfiltered/raw) on robustness.

hierarchical time–frequency representations using a feature extractor backbone from dynspecs and outputs binary decisions for class occurrence thresholded by a novel per-class decision thresholding technique explained in Appendix [X-6]. While inspired by architectures commonly used for object detection, the network is used purely as a global feature extractor and classification and does not perform spatial or temporal localization. From a physical standpoint, the learned representations correspond to characteristic burst morphology such as drifting emission lanes, broadband continua, and fine-structure clustering.

Training is performed using a multi-label loss function from equation [4] that treats each burst type independently, with additional emphasis placed on rare and ambiguous cases. This reflects the intermittent nature of solar radio activity and encourages the model to focus on weak or borderline burst signatures rather than being dominated by abundant quiet observations. As in the baseline model, strict data leakage controls are enforced through persistent train–validation–test splits.

At inference time, predicted probabilities are converted into binary decisions using class-specific decision thresholds learned from validation data. This accounts for systematic differences in score calibration between burst types and improves recall for physically significant but rare events without altering the model architecture.

Model performance is evaluated on a held-out test set using per-class precision, recall, and F1-score, allowing direct comparison with the baseline supervised model and enabling

assessment of the impact of synthetic data augmentation. Together, this framework provides a scalable and physically consistent approach to supervised solar radio burst classification, while highlighting the remaining limitations imposed by label noise and data scarcity that motivate the self-supervised methods introduced in the following section.

Results for Model 2: Figure 4 compares F1-scores across four experimental configurations designed to isolate the effects of (i) synthetic augmentation and (ii) domain shift between filtered and unfiltered (raw) dynamic spectra. Experiment 2 reproduces the training and evaluation conditions reported by Zhang et al. [?], where both training and testing are performed on datasets augmented with synthetic samples. Under these tightly matched conditions, performance appears uniformly strong across most burst types, illustrating how evaluation on synthetic data can lead to higher yet invalid performance estimates.

In contrast, Experiment 4 (train on filtered + unfiltered + synthetic, test on raw data) exposes the most severe performance degradation, particularly for Type 4, Type 5, and CasA. This setting most closely resembles realistic deployment, where raw observations contain strong background structure, calibration variation, and RFI. The pronounced drop across multiple classes indicates that neither supervised training nor synthetic augmentation alone is sufficient to learn representations that are invariant to such domain shifts.

Crucially, the failure mode observed in Experiment 4 highlights the need for representation learning in section V that explicitly enforces invariance across observational condi-

tions. This motivates the use of self-supervised pre-training, where the encoder is exposed to both filtered and unfiltered views of the same underlying event and is encouraged to learn domain-invariant, morphology-preserving features before classification. The following section therefore introduces a self-supervised learning framework designed to address the robustness limitations revealed by Experiment 4.

Detailed descriptions of data ingestion, network architecture, optimization procedures, threshold selection, and diagnostic metrics are provided in Appendix X.

V. DESIGNING SELF-SUPERVISED MACHINE LEARNING MODEL (MODEL 3)

While supervised learning provides a useful baseline for SRB classification, it relies heavily on labeled datasets. In practice, SRB archives contain a much larger volume of unlabeled observations that capture the full diversity of observing conditions, noise levels, and burst morphologies. Therefore, the study was also motivated to exploit this unlabeled data to improve classification robustness and reduce dependence on manual annotation. To achieve this, this paper adopts a novel Self-Supervised Learning (SSL) strategy, which is a representation-learning approach where the network is trained without expert-provided labels, by solving a “pretext” task that can be created directly from the images itself. In contrast to supervised learning (where the model must learn features and decision boundaries simultaneously from a limited labeled set), SSL aims to first learn a strong encoder that maps an input dynspec to a compact feature representation that is stable under realistic perturbations (noise, intensity scaling, partial occlusion, mild geometric distortions). This process is called “pre-training”. Once this encoder is learned, a smaller supervised model (a classification head) can be trained on top with fewer labeled samples to create cluster boundaries in the feature space. This is called “fine-tuning”.

This is particularly valuable for LOFAR SRB classification for two reasons. First, the available unlabeled dynspec archive is much larger than the labeled subset, so SSL allows the model to benefit from the full data distribution rather than only the annotated portion. Second, multi-label classification is inherently data-hungry: rare burst types or composite events have fewer clean real examples, and a supervised-only pipeline can overfit to superficial artifacts instead of learning burst morphology. By pretraining on a broad mixture of sources (labeled twins, unlabeled filtered, and synthetic dynspecs treated as unlabeled), the encoder is encouraged to learn general, transferable features that later improve label efficiency and robustness when fine-tuning for multi-label predictions.

What makes this SSL model fundamentally different from the supervised model is the training objective and the data usage pattern. The supervised model directly optimizes a multi-label BCE loss using equation [4] to predict class probabilities for a single image by reducing prediction error $\hat{y}_i - y_i$. The SSL model (in pre-training) instead optimizes a contrastive objective using equation [13] that does not require labels. It learns to *pull* together representations of two “views” of

the same 15-minute event and *separates* representations from different events. This makes the “encoder” a powerful version of “feature-extractor” sensitive to SRBs only. Conventional SSL models like SimCLR uses an image to create a “positive pair” by adding augmentations like noise, jitter, etc. In this case, the SRB “positive pair” is stronger than generic SimCLR because it includes paired filtered/unfiltered twins from the labeled twins split (see Section XI-A), which explicitly teaches the encoder invariance to RFI—exactly the kind of domain shift that matters when later evaluating on unfiltered “raw” data. This makes this paper’s version of SimCLR SSL model specifically tailored to LOFAR SRB classification task [?].

Another key methodological aspect is a guaranty of data leakage control: val/test labeled samples are never used during SSL pretraining or finetuning, neither in filtered nor unfiltered form. This keeps the SSL stage aligned with proper machine learning experimental protocol while still exploiting the large unlabeled pool. This means that the validation and test datasets are kept separate during the entire pre-training and fine-tuning pipeline, so the model never sees this data until required to keep the evaluation of the model as close as possible to a “real-life” test if the model is deployed.

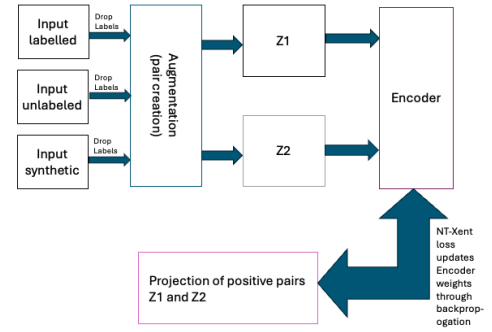


Fig. 5: Flow diagram representing SSL pre-training phase XI-A

A. Multi-source SSL Pre-training

To reduce dependence on scarce and noisy labels, the model is first trained using SSL on a large pool of unlabeled dynamic spectra. The goal of SSL pre-training is not to predict burst types, but to learn a feature representation that captures physically meaningful time–frequency morphology (e.g., drifting lanes, broadband continua, fine structure) while becoming robust to nuisance factors such as preprocessing differences and moderate corruption.

Pre-training data are drawn from three sources to maximize diversity and realism:

- 1) the training split of labeled observations, treated as unlabeled,
- 2) a large set of unlabeled LOFAR spectra, and
- 3) synthetic spectra, which are included as additional unlabeled examples to enrich the morphology distribution.

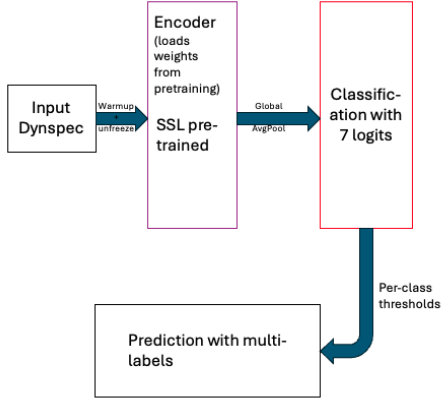


Fig. 6: Flow diagram representing SSL fine-tuning phase XI-B. Similar to supervised flow diagram 14, replacing YOLO feature extractor for the Encoder

To prevent leakage, only the training indices are used during SSL pre-training, while validation and test indices remain unseen until evaluation.

A key design feature is the use of “twin” views for a subset of samples: the same observation is available in filtered and unfiltered form. In these cases, the SSL “positive pair” is defined as filtered vs. unfiltered versions of the same event, explicitly teaching the encoder to become invariant to RFI mitigation and preprocessing domain shift. For other unlabeled samples (without a twin counterpart), two different views are generated through dynspec-safe augmentations that preserve burst morphology while introducing realistic variability.

During SSL pre-training, a convolutional encoder converts each input spectrum into a compact representation, and a small projection head maps this representation into an embedding space for contrastive learning. Training uses a standard contrastive objective (NT-Xent): two views of the same underlying observation are pulled closer in embedding space, while views of different observations are pushed apart. The practical output of this phase is a saved encoder weights that captures a morphology-aware representation suitable for classification.

Full details of the multi-source pretraining dataset construction, normalization, augmentation pipeline, encoder/projection architecture, and NT-Xent loss are provided in Appendix XI.

B. Multi-Label Classification SSL Fine-Tuning

After SSL pre-training, the learned encoder is reused as the feature extractor for supervised multi-label burst classification, using the same multi-hot label format as in the baseline supervised model (see Appendix XI-B). Fine-tuning learns a lightweight classification head on top of the encoder to map learned representations to per-class burst probabilities.

A strict and reproducible data protocol ensures fair comparison with the baseline supervised model and prevents leakage. Dataset splits are anchored to the original labeled split indices, and evaluation is performed on the same held-out test set as in the supervised experiments. Fine-tuning is intentionally

designed to test generalization beyond preprocessing artifacts: training uses the cleaner (filtered) training observations, while validation and test are performed on unfiltered observations, making the task closer to real operational conditions.

Where synthetic data are used, they are appended only to the training split, while validation and test remain purely real observations. This mirrors the evaluation discipline used in the supervised model and ensures that performance gains reflect improved generalization rather than familiarity with synthetic samples.

Fine-tuning optimizes a standard multi-label classification loss and outputs a vector of per-class probabilities. As in the supervised baseline, probabilities are converted into final multi-label predictions using per-class thresholds calibrated on the validation set, which helps account for class imbalance and differing score calibration across burst types. Performance is reported using the same set of metrics as the supervised model (per-class precision/recall/F1 and aggregate scores), enabling direct comparison of baseline supervised learning versus SSL-enabled learning.

Results for Model 3: Figure 7 shows that F1-score performance is high when evaluation is performed on filtered test data (Experiments 1–2), even as training conditions change (e.g., inclusion of synthetic samples in Experiment 2). However, the key stress test is Experiment 3, where the model is evaluated on raw (unfiltered) dynamic spectra. In this setting, several classes scored relatively higher than model 2 [fig. 4] indicating that the model 3 performs better than any other techniques on raw SRBs. This behavior directly motivates self-supervised learning (SSL) pre-training: instead of relying solely on supervised labels to learn features, SSL can expose the encoder to paired views of the *same* underlying event across filtering conditions and augmentations, explicitly encouraging invariance to nuisance factors while preserving morphology. In other words, SSL aims to learn a domain-invariant representation space *before* fine-tuning, so that classification task does not collapse when the input distribution shifts from filtered to raw observations, as seen in Experiment 3.

As a result, this paper’s novel SSL training procedure for SRB classification operates on a feature space that is already morphology-aware and noise-tolerant, allowing the classifier head to maintain decision quality even when the input distribution becomes harsher and closer to operational deployment. In practical terms, the SSL model demonstrates stronger generalization under realistic test conditions, validating the proposed technique as a scalable and deployment-relevant strategy for SRB identification and classification.

Full details of the fine-tuning architecture, staged training/unfreezing strategy, optimization settings, and threshold calibration procedure are provided in Appendix XI.

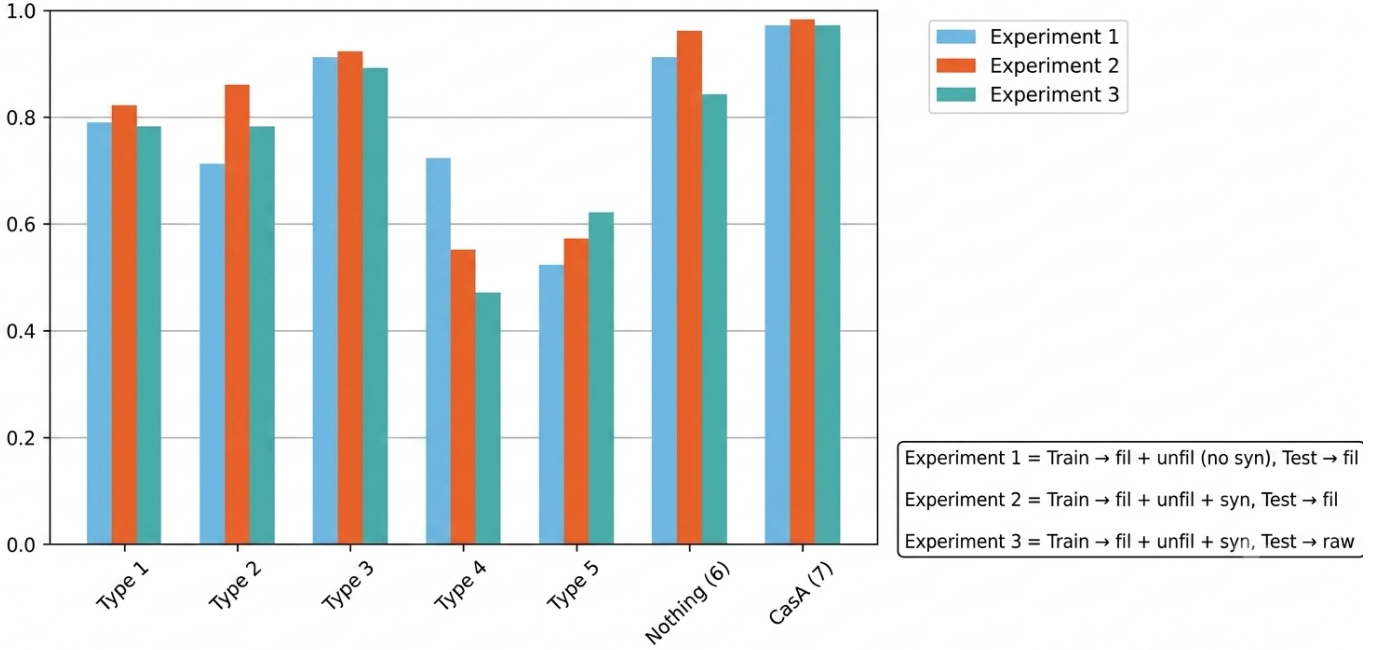


Fig. 7: Self-supervised (SSL) (model 3) Per-class F1- score performance across three configurations that progressively increase domain difficulty: training on filtered+unfiltered views (Exp. 1), adding synthetic augmentation (Exp. 2), and evaluating under raw (unfiltered) test conditions (Exp. 3). The results highlight which burst classes remain stable under domain shift and which degrade when moving to raw observations.

VI. DISCUSSION

This section reports how different modeling choices - detection (related research) versus classification, supervised versus self-supervised learning (SSL), filtered versus raw (unfiltered) test conditions, and the inclusion of synthetic augmentation - affect multi-label solar radio burst (SRB) recognition. The comparisons are consolidated in Table II, where the primary metric is taken as mAP for detection baselines from Mingming et al. [?], accuracy for single-label classifier from Zhang et al. [?], and micro-F1 ($\mu F1$) for multi-label classification methods (this research) where accuracy/mAP are not directly applicable. The goal of this evaluation is not only to report *how well* a method performs, but to explain *why* particular metrics are required for SRB data and *what* each experimental condition reveals about robustness under realistic observational variability [?], [?].

A key distinction among the compared approaches is the underlying task formulation. Bounding-box (BB) detection methods, including Task-Aligned One-Stage Object Detection (TOOD) style pipelines, typically measure performance using mAP, which evaluates both localization and detection quality [?], [?]. This is valuable when reliable bounding boxes exist and bursts can be treated as object-like instances. However, BB annotation is costly and ambiguous for SRBs because burst boundaries in time-frequency space can be diffuse, drifting, overlapping, or texture-like rather than compact objects. Extended lanes and broadband continua of a Type IV SRBs are not naturally captured by a single "object box", and multi-

burst spectra require multiple boxes with consistent overlap rules and labeling conventions [?], [?]. These factors make BB supervision harder to scale and potentially less aligned with the practical objective of automated cataloguing at the spectrum level. Multi-label classification instead directly predicts which burst types are present in a 15-minute LOFAR window, naturally supporting co-occurrence without requiring spatial annotations and therefore better matches the workflow goal of searchable event tagging.

Literature single-label baselines, such as Zhang et al. [?], reported here as Learmonth (L) and Culgoora (C), to quantify performance under instrument and observing-condition shifts [?], [?]. The practical motivation for reporting separate L and C accuracies is that radio burst appearance varies across instruments and observing conditions, and performance under domain shift is often the limiting factor in real deployment [?]. In Table II, Zhang et al. [?] report $\text{Acc}(L)=0.8747$ and $\text{Acc}(C)=0.8659$ under their Syn + L + C setting [?]. These results provide context, but they are not directly comparable to LOFAR multi-label performance because the task is single-label and the data distribution differs from the LOFAR 15-minute windows used in this research. Additionally, a similar methodology of Zhang et al. [?], was implemented in Supervised test condition where synthetic data was also included in the test dataset. The results for that experiment gave micro-F1 score even better than the derived from research. But a major hole in this approach was revealed from per-class threshold metric. It reveals that even though the model score on test

TABLE II: Main performance comparison across literature baselines and proposed approaches. Primary metric is reported as mAP (detection), accuracy (classification), or micro-F1. SSL outperforms every other technique. Per-class thresholds are listed in class order {Type1, Type2, Type3, Type4, Type5, Nothing(6), CasA(7)} In general, high threshold means the model has higher confidence for getting a correct prediction for that class. Detailed analysis is found in Appendices X, XI

Approach	Test condition	Primary	μP	μR	$\mu F1$	M-P	M-R	M-F1	Subset Acc	Per-class thresholds (T1,T2,T3,T4,T5,N6,T7)
[?] (BB)	Detection (BB, paper)	mAP=0.7990	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
[?] (single-label classifier)	Syn + L + C (paper)	Acc(L)=0.8747, Acc(C)=0.8659	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Baseline supervised	Unfiltered (raw)	Acc=0.8794	N/A	N/A	N/A	0.8312	0.7541	0.7745	N/A	N/A
Supervised (no syn)	Filtered	$\mu F1=0.8392$	N/A	N/A	0.8392	N/A	N/A	0.4162	0.7188	[0.275, 0.396, 0.486, 0.523, 0.465, 0.403, 0.338]
Supervised (fil+syn)	Filtered + Syn (similar to [?])	$\mu F1=0.9013$	N/A	N/A	0.9013	N/A	N/A	0.3543	0.8219	[0.336, 0.313, 0.429, 0.429, 0.283, 0.448, 0.316]
Supervised (fil+syn)	Filtered	$\mu F1=0.8100$	N/A	N/A	0.8100	N/A	N/A	0.4045	0.6828	[0.292, 0.385, 0.430, 0.473, 0.304, 0.444, 0.466]
Supervised (fil+unfil +syn)	Unfiltered (raw)	$\mu F1=0.4643$	N/A	N/A	0.4643	N/A	N/A	0.3160	0.0097	[0.134, 0.741, 0.106, 0.050, 0.058, 0.502, 0.050]
SSL (no syn)	Filtered	$\mu F1=0.8610$	0.8367	0.8868	0.8610	0.7557	0.8497	0.7905	N/A	[0.85, 0.75, 0.90, 0.90, 0.90, 0.55, 0.45]
SSL (+syn)	Filtered	$\mu F1=0.8986$	0.8958	0.9013	0.8986	0.8537	0.8291	0.8397	0.7835	[0.75, 0.80, 0.90, 0.80, 0.80, 0.70, 0.80]
SSL (+syn)	Unfiltered (raw)	$\mu F1=0.8500$	0.8126	0.8909	0.8500	0.7425	0.8223	0.7672	0.6921	[0.60, 0.75, 0.60, 0.20, 0.70, 0.75, 0.80]

dataset was incredible, the model seemed to not show higher confidence in its predictions, since low per-class threshold is due lower model prediction probability of a predicted class.

In multi-label evaluation, micro (μ)- and macro (M)- metrics capture different operational priorities. Micro-averaged metrics pool decisions across all classes and samples, reflecting overall system performance and being influenced more strongly by frequent classes; this is why micro-F1 is a useful primary metric for end-to-end cataloguing behavior. Macro-averaged metrics compute per-class scores and average them equally, making macro-F1 a better indicator of minority-class performance and a fairer summary under imbalance. Subset accuracy is also reported for some methods, measuring exact match of the entire label set per spectrum. This is a strict metric that becomes challenging in multi-burst cases because a prediction that is mostly correct but misses one label is still counted as incorrect. Subset accuracy therefore acts as a complete-labelling indicator rather than a general performance measure in mixed-activity spectra [?].

Another practical component in SRB multi-label classification is the use of per-class decision thresholds. A single global threshold (e.g., 0.5) often leads to systematic over- or under-prediction because classes differ in prevalence, ambiguity, and probability calibration. The thresholds listed in Table II (in the fixed class order {Type1, Type2, Type3, Type4, Type5, Nothing(6), CasA(7)}) convert probabilities into final binary decisions and are tuned to improve F1 behavior per class. This becomes particularly important when models are used for real catalogue generation, where the operational cost of false positives and false negatives differs across burst types and scientific use-cases [?].

The results in Table II show that supervised learning is highly sensitive to train/test conditions and domain mismatch. The baseline supervised model tested on unfiltered raw spectra reports Acc=0.8794, yet macro-F1 is 0.7745, illustrating how accuracy can mask uneven per-class behavior under imbalance. Under filtered testing conditions, the supervised model without synthetic data reaches $\mu F1=0.8392$, with subset accuracy 0.7188, but macro-F1 remains much lower (0.4162), consistent with harder minority classes and ambiguous morphologies.

Adding synthetic data in a purely supervised pipeline does not automatically improve generalization: the supervised (filtered + synthetic) run decreases to $\mu F1=0.8100$ (macro-F1=0.4045), which is consistent with the possibility that synthetic samples introduce a distribution shift that slightly degrades test performance when the feature extractor is trained only under label-driven objectives [?], [?]. The most severe failure appears when supervised training mixes conditions and is evaluated on raw unfiltered spectra: trained on mix (filtered + unfiltered + synthetic) and tested on unfiltered "raw data" achieves $\mu F1=0.4643$ with subset accuracy 0.0097, indicating a strong domain mismatch where probability calibration and multi-label decision consistency break down under raw observational noise and interference patterns [?], [?].

In contrast, SSL produces consistently stronger and more balanced performance, supporting the conclusion that representation learning improves robustness for SRB morphology in alternative studies [II]. SSL (no synthetic) tested on filtered reaches $\mu F1=0.8610$ with $\mu P=0.8367$, $\mu R=0.8868$, and macro-F1=0.7905, indicating both high overall performance and substantially improved equal-weight behavior across burst types. When synthetic data is included after SSL, the best classification result is achieved: SSL (+syn) on filtered reaches $\mu F1=0.8986$ with $\mu P=0.8958$, $\mu R=0.9013$, macro-F1=0.8397, and subset accuracy 0.7835. This pattern suggests that SSL pretraining stabilizes the feature space so that synthetic augmentation becomes net-positive by increasing coverage of under-represented morphologies rather than destabilizing the decision function [?], [?]. Importantly, SSL also maintains strong performance under raw unfiltered conditions: SSL (+syn) tested on unfiltered achieves $\mu F1=0.8500$ with subset accuracy 0.6921, demonstrating much better resilience to domain shift than the supervised alternatives under similar test conditions [?]. Even though the subset accuracy is a bit lower here, Figure [8] clearly demonstrates that model has learned robust solar radio burst features to even give correct predictions on *weak and/or incorrect labels*. The metrics like subset accuracy and precision drops because cases of weak and/or incorrect labels as inspected in figure [8], since model performance metrics are measured using these labels. But

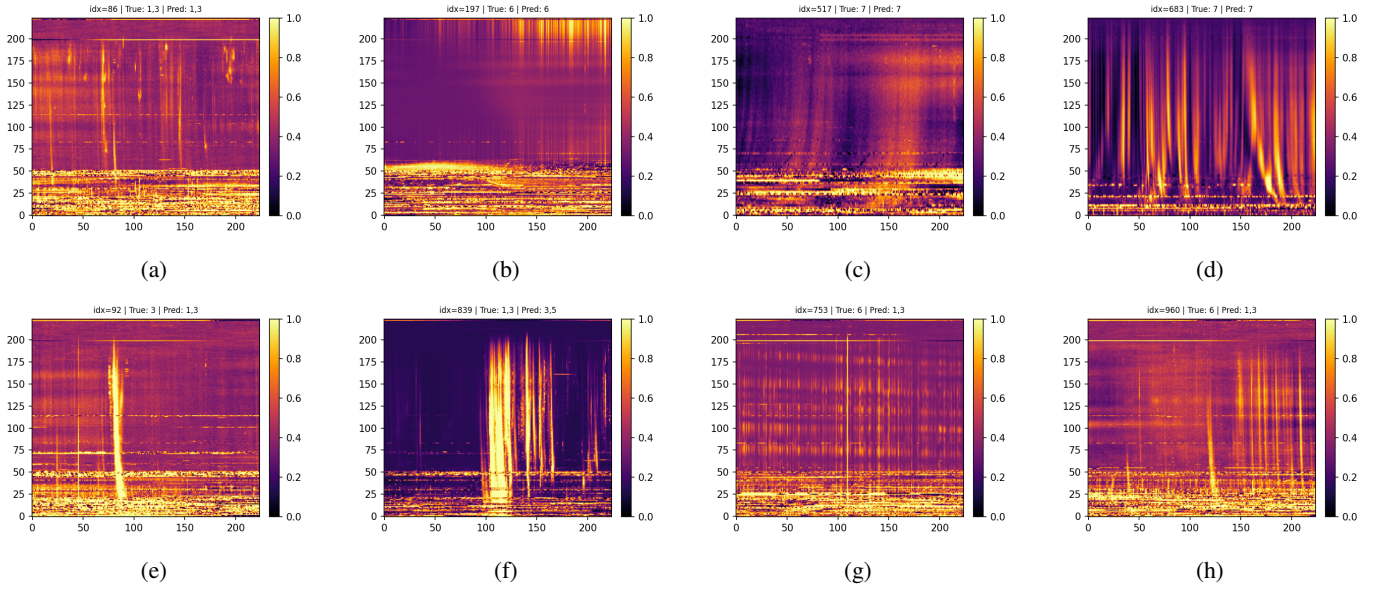


Fig. 8: A few examples of Self-Supervised model Predicted vs True Label visualization on test data: Images (a) to (d) are classified correctly. Images (e) and (f) have predicted additional labels to what's already known. Image (g) is a wrong prediction and interestingly, image (h) is technically a wrong prediction but after visual inspection, the image actually contained the predicted bursts indicating the model has learned the correct features and it predicted correct labels even when the true labels are incorrect.

for a test dataset, model only sees the labels and measure performance after the prediction has been done. So, even though the metric scores drop, the SSL model has learned SRB features well at the pre-training phase such that incorrect and/or weak labels are not affecting it in fine-tuning stage much to make give wrong predictions on the test data.

Overall, the experimental comparisons highlight four practical takeaways. First, the evaluation metric must match the SRB problem structure: accuracy is insufficient under imbalance and multi-label co-occurrence, while micro- and macro-F1 together describe both system-level performance and minority-class reliability. Second, supervised pipelines are highly sensitive to the appearance statistics induced by filtering versus raw data, and they can fail sharply under domain shift and interference variability [?], [?]. Third, SSL reduces this sensitivity by learning representations that emphasize stable burst morphology under diverse augmentations and heterogeneous unlabeled inputs, improving both micro-F1 and macro-F1[?]. Fourth, synthetic augmentation is most beneficial when combined with SSL, where the learned representation is robust enough to absorb synthetic variability without collapsing calibration [?], [?]. In a deployable automatic SRB classification setting, these behaviors matter directly: high recall prevents missing rare scientifically valuable events, high precision avoids overwhelming catalogues with false alarms, macro-F1 reflects fairness across burst types, and subset accuracy reflects how often a spectrum is fully and correctly labelled for multi-burst analysis [?].

This research shows that improving the data side (better labels, consistent HDF5 structuring, orientation/RFI mitiga-

tion, and synthetic augmentation for rare classes) gives a clear performance jump, but only up to a point. After that, gains start to plateau because the remaining errors are dominated by representation limits and distribution shift (especially when moving between filtered and unfiltered spectra). At that stage, the biggest boost comes from advanced machine learning strategies, specifically self-supervised learning (SSL).

By pretraining on large, ready-to-use unlabeled LOFAR archive data (plus train-only fil/unfil twins and optional synthetic spectra) and then fine-tuning on the labeled split, SSL learns a stronger burst-aware feature representation than supervised-only training. Practically, this reduces reliance on time-consuming manual labeling for supervised models which requires thousands of labels to train [?], [?], [?]. Automatic labeling technique from section VIII-C2 enables limited label generation sufficient for SSL fine-tuning which has increased robustness and making performance strong enough to be considered deployable for automated triage and cataloging support. These experiments show that SSL pretraining followed by supervised fine-tuning yields stronger and more data-efficient multi-label classification than training from scratch. Under filtered testing, the best configuration (SSL + synthetic augmentation) achieved micro-F1 = 0.8986 (macro-F1 = 0.8397), outperforming supervised baselines under comparable conditions. Importantly, the SSL model also remained robust on unfiltered (“raw”) test data with micro-F1 = 0.8500 (macro-F1 = 0.7672), indicating improved generalization under noise and interference shifts.

Evaluation emphasized metrics that match the SRB problem. Accuracy is unreliable for imbalanced multi-label test sets

because dominant classes can inflate it while minority bursts are missed. For this reason, this research reports precision/recall/F1, with micro-F1 reflecting overall decision quality and macro-F1 highlighting performance on rare burst types. Per-class thresholding further improves practicality because each burst class has different score distributions and base rates.

VII. CONCLUSION AND FUTURE SCOPE

This study presents an end-to-end AI-driven framework for the automated identification and multi-label classification of solar radio bursts (SRBs) in LOFAR dynamic spectra. The framework is designed to operate on realistic 15-minute observing windows and to assign physically meaningful burst labels (Type I–V), while also identifying quiet intervals (“Nothing”) and calibration observations involving Cassiopeia A (CasA). The overarching objective has been to move beyond small, curated examples toward a scalable and deployable pipeline capable of supporting large solar radio archives and operational triage.

A central finding of this work is that performance improvements arise from both data-centric and model-centric advances, but in different regimes. Early gains are achieved by improving the data foundation: physically motivated label expansion from NOAA event reports, consistent HDF5 structuring, orientation correction, RFI mitigation, and targeted synthetic augmentation for rare burst classes. These steps yield a clear and measurable improvement over simplistic supervised baselines. However, the results also show that such improvements plateau once errors become dominated by representation limits and distribution shift, particularly when models trained on filtered spectra are evaluated on unfiltered (“raw”) observations.

At this stage, the most significant performance gains are achieved through advanced machine learning strategies, specifically self-supervised learning (SSL). By pretraining on large volumes of unlabeled LOFAR data—augmented with filtered/unfiltered twin observations and optional synthetic spectra—and subsequently fine-tuning on a limited labeled subset, SSL enables the model to learn burst-aware, morphology-driven representations that are more robust to noise, preprocessing differences, and label uncertainty than representations learned through supervised training alone.

Practically, this reduces reliance on large, manually labeled datasets, which have traditionally been a bottleneck in solar radio burst classification. Whereas prior supervised approaches typically require thousands of carefully annotated examples, the automatic labeling strategy developed in this work enables sufficient label generation for effective SSL fine-tuning, resulting in models that are both data-efficient and operationally realistic. Experimental results confirm that SSL pretraining followed by supervised fine-tuning consistently outperforms training from scratch, particularly for rare and ambiguous burst types.

Future work can build on this foundation in three concrete directions:

- 1) Burst localization on top of classification: extend from “which types exist in this 15-minute window?” to “where in time–frequency does each burst occur?”, enabling event-level products (tracks/regions) rather than only window-level tags.
- 2) Deployment-ready integrated pipeline: package the full workflow (data ingestion → preprocessing → inference → reporting) into a robust, reproducible system that can be integrated into operational environments, such as ASTRON computing workflows, TU Eindhoven CubeSat project and other platforms where automated burst screening is valuable.
- 3) Explainability (XAD): develop interpretable outputs (saliency/attribution maps, class-specific evidence regions, uncertainty reporting) so scientists can validate why a prediction was made—critical for trust, debugging, and scientific usability.

REFERENCES

VIII. APPENDIX: DATA AND PRE-PROCESSING IN DETAIL

This section is related to Section III describes data to be classified in more details, and presents detailed process of LOFAR data acquisition and dataset construction. The labeling strategy, including the integration of initially provided labels and externally derived labels is also presented. Finally, orientation correction and radio-frequency interference (RFI) mitigation procedures are described, which are critical for ensuring data quality prior to machine learning model training.

A. SRBs and other images

This work uses LOFAR dynamic spectra of SRB data represented as time–frequency intensity matrices, where the horizontal axis corresponds to time and the vertical axis corresponds to observing frequency. Below, each class is described by its pattern in a dynamic spectrum, because the learning problem is ultimately pattern recognition on these shapes.

- 1) Type I (noise-storm bursts): Type I activity usually appears as many small, bright, short-lived bursts clustered in time, often concentrated in a relatively narrow frequency region. It often looks like a busy patch or cloud of flickering points/short streaks that can persist as stormy activity. This type is commonly associated with active regions and ongoing electron acceleration, so it can be frequent and repetitive rather than a single isolated event [?].
- 2) Type II (shock-related emission): Type II bursts are commonly linked to shock waves propagating through the corona (often driven by eruptive events). In dynamic spectra they tend to form slowly drifting bright lanes: the emission shifts from higher to lower frequencies over minutes, producing a slanted band that moves gradually. It is also common to see a pair of lanes (interpreted as fundamental/harmonic structure), making it look like two roughly parallel tracks [?].
- 3) Type III (electron-beam bursts): Type III bursts are produced by fast electron beams escaping along magnetic

field lines. Visually, they are the “fast streaks”: bright features that drop from high to low frequency very quickly (often seconds to tens of seconds), so they appear as steep diagonal lines across the plot. When multiple electron injections occur, many near-parallel fast streaks can appear in a short period [?] [?].

- 4) Type IV (broadband continuum, trapped/extended electrons): Type IV emission is typically a more extended radio “glow” that can last much longer than Type III. In the spectrum it often shows up as a broad, sustained bright region (a continuum) covering a wider frequency range, sometimes with embedded structure, but the key visual cue is persistence: it does not just flash and disappear. This class often corresponds to larger-scale coronal configurations in which energetic electrons remain confined or continuously replenished, so the signature can look like a long bright blanket rather than a single line [?].
- 5) Type V (continuum following Type III activity): Type V is commonly described as a short-lived continuum that occurs in connection with Type III bursts—conceptually, an “afterglow” that follows a period of Type III streaks. In the spectrum it often looks like a diffuse bright region that turns on after (or alongside) Type III activity and persists briefly rather than forming a single sharp drifting line [?].
- 6) Empty / quiet spectrum (“Nothing”): This class represents intervals where no recognizable burst morphology is present. In practice, these frames can still contain background variations, instrument effects, and interference but lack the coherent burst-like structures that define Types I–V. Treating this as an explicit class is important because it prevents the classifier from being forced to “hallucinate” a burst label when the correct outcome is “no event of interest.”
- 7) Cassiopeia A calibration spectra (Cas A): Cas A frames occur because radio-telescope pipelines frequently observe bright, well-characterized radio sources for calibration and quality control, and Cassiopeia A is one of the standard bright calibrators used in low-frequency radio astronomy. These observations can be strongly source-dominated and can look systematically different from SRB spectra; if they are not separated, they can contaminate the SRB training set and distort decision boundaries.

All data are stored using the Hierarchical Data Format version 5 (HDF5) or (.HDF5 format), a binary file format designed for efficient storage and access of large, structured scientific datasets. HDF5 supports hierarchical grouping of multiple arrays and metadata (called keys) within a single file, enabling consistent organization of intensity matrices, timestamps, frequency and time ranges, and labels. This format allows memory-efficient partial loading, which is essential for large-scale machine learning workflows and high-performance computing environments. Each record is stored in an HDF5 container dataset with the following keys:

- 1) **data**: a floating-point matrix of shape 800×500, where the x-axis corresponds to time (15-minute observation window) and the y-axis corresponds to frequency (10–240 MHz).
- 2) **timestamps**: a string of unique identifiers encoding observation ID, date, and start time.
- 3) **freq_range**: a two-element vector specifying the lower and upper frequency bounds (in MHz) of the observation.
- 4) **time_range**: a two-element vector indicating the start and end time of the 15-minute window.
- 5) **indices**: integer indices mapping each sample to its position in the original acquisition order.
- 6) **labels**: an integer or multi-hot encoded vector (shape 7C, where C is the number of classes) representing the assigned burst class(es).

This representation of the dataset remains consistent throughout the entire methodology for data saving and usage.

B. LOFAR data acquisition and HDF5 dataset construction

The LOFAR dataset was obtained via the ASTRON space-weather service API, which provides per-observation metadata and file URLs for compressed solar beam-formed dynamic spectra products. The acquisition pipeline proceeds as follows:

1) *Querying the ASTRON API and building the download manifest*: A paginated API query returns a list of observation tasks after a specified start date. For each task result, the pipeline constructs a unique key:

$$timestamp/key = obsID_date_time \quad (6)$$

and extracts the corresponding *.fits* file URL from *additional_files_url*, in the web API and the associated *freq_range* and *time_range* metadata fields. The collected (key, url) pairs are exported to a CSV manifest to ensure reproducible downloads.

2) *FITS ingestion, normalization, and resizing*: For each *.fits* file, the intensity matrix is loaded and normalized using a robust background estimate computed from the 10–30% quantile band along the time axis (as implemented by sorting values and averaging a mid-lower percentile slice). The normalized matrix is then scaled using percentile-based contrast bounds (20th–80th percentile) followed by a logistic mapping (sigmoid) and resized to a fixed spatial resolution (500×800) for consistent ML input. Note that the downloaded image here are flipped and rotated with time axis (500) and frequency axis (800). this work will later fix the orientation in VIII-D.

The full unlabeled corpus is written efficiently into a memory-mapped array (.mmap) and finally stored in HDF5 as /data, /timestamps, /freq_range, /time_range. This process results in the bulk unlabelled HDF5 dataset used for Machine Learning training in IV and for deriving labeled subsets.

C. Labeled dataset construction and label sources

Labeled data used in this work comes from two sources: (i) ASTRON-provided labels and (ii) labels generated using NOAA event reports.

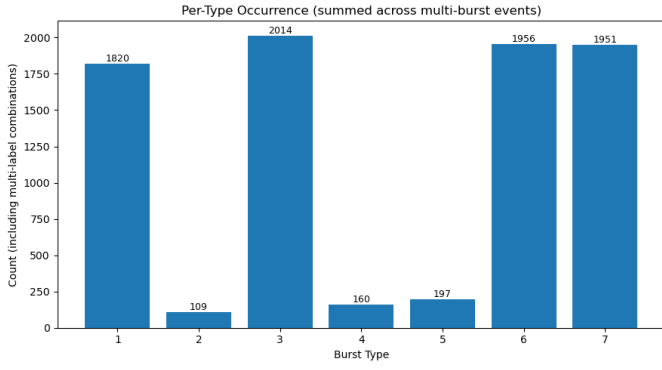


Fig. 9: Per class distribution after automatic label generation from section VIII-C2

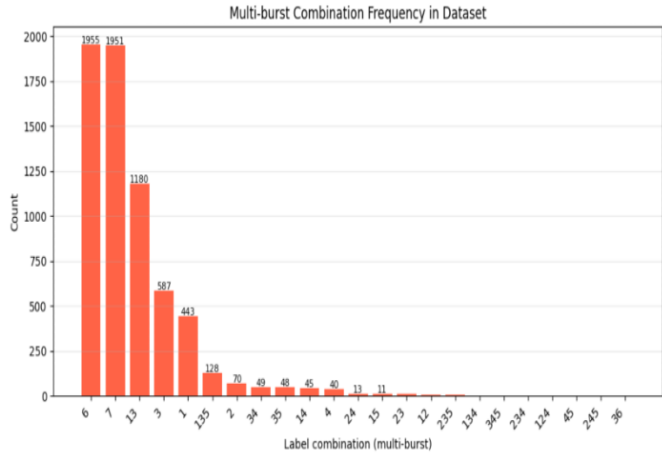


Fig. 10: Multi-Label burst distribution after automatic label generation from section VIII-C2

1) Initial labeled set (expert-provided, 2,854 samples):

An initial labeled set of 2,854 LOFAR dynamic spectra was provided by ASTRON in the form of a CSV containing timestamps/keys and labels. To create a labeled HDF5 subset, the pipeline:

- 1) loads the bulk downloaded HDF5 dataset (unlabeled)
- 2) matches the CSV timestamp keys to the HDF5 timestamps
- 3) extracts the corresponding data and metadata rows
- 4) writes them into a labeled HDF5 file (preserving the same key structure and metadata fields, adding *labels* key)

This subset serves as the initial supervised anchor dataset for baseline model IV-B prior to label expansion.

2) *Label generation using NOAA solar event reports (expansion to 6,555 labels):* Since initial labels are not enough for all different burst types for machine learning processes, there was a need to create a large labeled corpus. To expand labels beyond the initial 2,854 samples, daily NOAA Solar Event Reports are used as a structured external catalog. The goal is to map cataloged radio events to LOFAR 15-minute dynamic spectrum windows, generating additional labels in

a reproducible way. For each date in the target interval, the pipeline downloads the NOAA daily event report text file using a deterministic URL pattern. Each file is parsed using fixed-width columns derived from the header line, extracting:

- event times: *begin, max, end*
- event type: *Type*
- frequency field: *Loc/Frq*
- burst descriptor: *Particulars*

Only entries with *Type* == "RSP" are retained, and only those whose *Particulars* begins with a Roman numeral corresponding to burst types I–V are considered for labeling. The reports can contain placeholder patterns (e.g., *///*) that shift digits across columns. A deterministic post-processing step normalizes event times into machine-parsable *HHMM* strings (e.g., truncating *begin* to 4 digits; reconstructing *end* to 4 digits). This produces a cleaned JSON catalog used for label assignment.

Event-to-spectrum alignment: The labeling algorithm formalizes labeling as an "overlap problem" between NOAA event intervals and LOFAR 15-minute timestamp bins.

1) Timestamp bin construction

- Each LOFAR spectrum corresponds to an interval:
 - *start_interval* = *date* + *time*
 - *end_interval* = *start_interval* + 15minutes

2) Event time construction

- NOAA *begin* and *end* (HHMM) are converted into datetimes:
 - *begin_dt* = *date* + *begin_time*
 - *end_dt* = *date* + *end_time*

3) Label Assignment rule

- For each event, LOFAR intervals are labeled if they overlap the event window, with a special deterministic rule for short and long-duration events:
 - Case A: duration ≤ 75 minutes Label all LOFAR 15-min intervals that overlap the event:

$$\begin{aligned} & \text{event_begin} < \text{interval_end} \\ & \wedge \text{event_end} > \text{interval_start} \end{aligned} \quad (7)$$

- Case B: duration > 75 minutes (long events) Identify the 15-min interval containing the event start, then label the next 6 consecutive intervals (i.e., up to 90 minutes coverage). This bounds label propagation for very long catalog events to a fixed number of LOFAR windows and prevents over-labeling.

Event *Particulars* is mapped from Roman numerals to integer labels:

$$I \rightarrow 1, II \rightarrow 2, III \rightarrow 3, IV \rightarrow 4, V \rightarrow 5$$

Integration of additional Type II list: An additional Type II labels provided by ASTRON as csv file is converted into LOFAR timestamp key format and matched against the LOFAR timestamps table. Matched timestamps are assigned

label 2 and merged with NOAA-derived labels and the initial 2,854 expert-made labels.

Calibration CasA spectrum handling: The downloaded bulk data also consisted observations from the calibration source of LOFAR [1]. This source is a supernova remnant called Cassiopeia - A. Cassiopeia - A (Cas A) calibration spectra are included because the LOFAR solar observing pipeline intermittently records observations dominated by this bright supernova remnant, which is routinely used as a flux calibration source. These observations are stored in the same data format and retrieved through the same acquisition pipeline as solar observations, making them indistinguishable at the raw-data level. Treating Cas A as a separate class prevents misclassification and ensures that the learning models do not incorrectly interpret calibration data as solar radio bursts. Since the NOAA labelling process is completely autonomous, it wrongly labeled some CasA data as a solar radio burst. This wrong labeling could be due to the fact that the process uses a time-range (eg. < 75 minute window) to label solar radio bursts. So, when manually inspected these labeled samples, correction of these these wrong labels was done manually to unsure that the machine learning model only train on correct labels for the images. While correcting these labels, it was found out that some images had multi-bursts in one 15 minute image spectrum. Therefore, manual assignment of multi-labels was done to those images. These multi-labeled images will be fed to the machine learning model in the later sections thus formulating an innovative multi-label classification system which is a novel approach in this field.

After combining (expert-made labels + NOAA labels + external Type II list) and applying validity/duplicate filtering (Section VIII-D), the total labeled set reaches **6,555 labeled samples**. 9 10

D. Orientation fix and RFI Mitigation

Raw LOFAR dynamic spectra exhibit orientation and background variations arising from instrumental configuration, calibration procedures, and terrestrial interference. From a physical standpoint, these effects obscure transient coronal emission features by introducing slowly varying intensity structures unrelated to solar radio processes.

The downloaded bulk images were flipped and rotated as seen in figure 11. Before applying RFI filtering to it, this work fix the orientation from (500x800) frequency vs time to (800x500) time vs frequency.

This reseach uses modified version of Salmane et al. [?] pipleine to reduce radio-frequency interference (RFI) and non-stationary background fluctuations in LOFAR dynamic spectra. this work adapted the denoising strategy of [?], consisting of (i) background/RFI mitigation followed by (ii) median filtering to suppress residual artifacts while preserving burst morphology. In Salmane et al. study, the background-reduced spectrum ($I_r(f, t)$) is computed from the input spectrum ($I_c(f, t)$) by subtracting either a short-term local mean ($I_b(f, t)$) or a long-term per-frequency baseline ($I_r(f)$), depending on whether (I_b) falls below a variability threshold

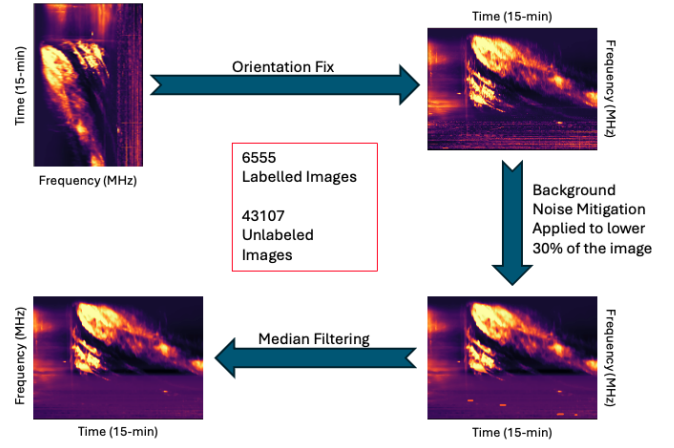


Fig. 11: Radio Frequency Interference (RFI) removal pipeline inspired by Salmane et al. [?]

($I_r(f) + k, r_r(f)$) (with ($k \in [1, 3]$) in the reference study). The method is expressed as

$$I_r(f, t) = \begin{cases} I_c(f, t) - I_b(f, t), & I_b(f, t) \leq I_r(f) + k r_r(f) \\ I_c(f, t) - I_r(f), & I_b(f, t) > I_r(f) + k r_r(f) \end{cases} \quad (8)$$

where ($I_c(f, t)$) denotes the input intensity at frequency bin (f) and time bin (t). The short-term background ($I_b(f, t)$) is computed as a running mean over a window (T_{short}) along time, while the long-term baseline ($I_r(f)$) and variability ($r_r(f)$) are computed per frequency row (robustly using the median and ($1.4826 \cdot MAD$) in the implementation). The parameter (k) controls the decision threshold between subtracting (I_b) or (I_r). Since each LOFAR sample in the dataset corresponds to a single 15-min frame, this study computed (I_b) using ($T_{\text{short}} \in [30, 45]$)sec (converted to columns via ($\Delta t = 900/W$ seconds per column), and estimate ($I_r(f)$) and ($r_r(f)$) from the same 15-min frame rather than the multi-hour context used in the reference work. To reflect stronger contamination at low frequencies and to avoid introducing seams, the subtraction is applied only to the lower band using a smooth cross-fade weight ($w(f)$) (cosine ramp centered at ($f_{\text{cut}} = 35\text{MHz}$) with half-width ($\Delta f = 3\text{MHz}$), yielding a weighted form ($I_r(f, t) = I_c(f, t) - w(f, \Delta(f, t))$), where ($\Delta(f, t) \in I_b(f, t), I_r(f)$) is selected by (8). Finally, to reduce the risk of attenuating bright burst structures, this work add a burst-protection override that prevents using the (I_b) subtraction when ($I_c(f, t)$) is well above the baseline and cap the subtraction magnitude to a fixed fraction of the pixel intensity (e.g., ($\Delta(f, t) \leq 0.6, I_c(f, t)$)). Residual impulsive artifacts are then suppressed by median filtering: while Salmane et al. study applies a direction-aware median filter guided by local gradients (Appendix IX), this work implement an efficient 2-D median filter over a ($(k_f \times k_t)$) neighborhood 'median_filter', selecting kernel sizes based on burst morphology (smaller kernels for thin/fast structures such as Type III; larger kernels for broadband continua such as Type IV).

IX. APPENDIX: SYNTHETIC DATA GENERATION IN DETAIL

This section fully documents Synthetic Data Generation procedure from section IV-C.

1) *Generator*: The generator G is trained to produce a time–frequency dynamic spectrum $\hat{x} = G(z, y)$ conditioned on a target label y , where z is a random latent vector (noise). Conditioning is implemented via a learned label embedding that is concatenated with z , after which a fully-connected projection reshapes the fused representation into a small spatial tensor and progressively upsamples it using transposed convolutions. This cDCGAN upsampling allows G to learn how to map latent factors to structured spectrogram patterns rather than relying on handcrafted simulation rules [?]. During training, the generator is optimized primarily through an adversarial objective: it receives gradients that encourage generated samples to be judged as “real” by the discriminator for the intended label. In practice, this means G is pushed to reproduce label-consistent morphology (e.g., coherent burst ridges, broadband continua, or fine structures depending on class definition) while maintaining diversity across samples generated from different z .

2) *Discriminator*: The discriminator D is trained to distinguish real samples x from generated samples $\hat{x}$, also conditioned on the label y . The discriminator uses strided convolutions for learned downsampling (rather than deterministic pooling), aligning with standard DCGAN principles for stable feature learning [?]. In addition, conditioning is implemented using a projection-style mechanism: D learns class-dependent feature alignment by combining an embedding of y with intermediate feature vectors. This encourages D to verify not only “realism” but also whether the sample is compatible with its asserted class. To reduce common GAN failure modes (e.g., mode collapse and training oscillations), the discriminator is regularized using a gradient-based penalty on real samples (R1-style regularization), a common stabilization approach in adversarial training that discourages overly sharp decision spikes in loss function and improves training robustness [?].

3) *Adversarial Training*: Training alternates between updating D (improving real/fake separability) and updating G (improving the realism and label-consistency of generated data $\hat{x}$). With hinge adversarial loss using in [?], it is not expected that generator and discriminator losses decrease monotonically; instead, they typically fluctuate and compete as each network adapts to the other. A discriminator that becomes too strong can temporarily raise the generator loss, while a generator improvement can temporarily increase discriminator error as seen in Fig. [12]. In this work, the training loop periodically checkpoints the generator and exports a fixed set of class-balanced synthetic samples, enabling selection of a suitable epoch for producing the final synthetic HDF5 corpus used in later supervised multi-label training. The intended outcome is a label-conditioned generator that can produce additional minority-class samples with realistic spectrogram structure, thereby increasing the diversity and quantity of training data available to the subsequent classifier [?].

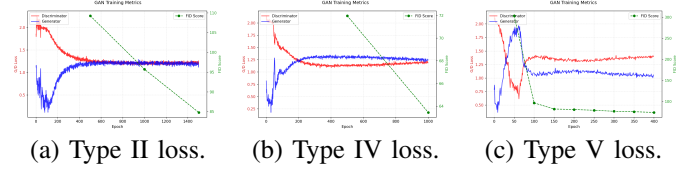


Fig. 12: cDCGAN training dynamics (G/D losses) and FID for minority-class synthesis.

4) *Quality assurance*: After training, quality of generated data samples was assessed using (i) visual inspection of generated dynamic spectrum over specific training epochs, and (ii) a distributional similarity metric called Fréchet Inception Distance (FID), computed from Inception-v3 feature embeddings of real and generated images [?]. When comparing FID against values reported in other SRB GAN studies, this study scores lower even when samples are visually reasonable because the score is computed in a feature space that depends strongly on the image statistics and acquisition domain. Here the input images also included unfiltered images for the minority classes [?], [?]. For this reason, FID was treated primarily as a within-dataset model selection indicator across checkpoints (trend and relative ranking), while visual inspection plausibility and label-conditional consistency served as additional acceptance criteria for final synthetic data sample export.

5) *HDF5 export*: After training, the generator produces exactly 1500 synthetic samples per requested label set (for, Type 2, Type 4, Type 5), which are resized back to the original dataset resolution. The synthetic samples are written to an HDF5 file with indices = 0..N-1, labels stored as the original label IDs, and metadata fields (freq_range, time_range, timestamps) copied or sampled from the source dataset to maintain structural compatibility of the dataset with the existing preprocessing and dataloading code (ref section VIII). This design allows synthetic data to be appended to the training split only in the subsequent multi-label classifier experiments, while validation and test sets remain purely real observations.

X. APPENDIX: MULTI-LABEL SUPERVISED CLASSIFICATION MODEL IN DETAIL

This section fully documents the Multi-Label Supervised Classification Model used in (section IV-D).

1) *Data ingestion, label parsing, and sample filtering*: The model inputs LOFAR dynspec images stored in a single HDF5 file created after labeled dataset generation described in Appendix refsec:appendix a, under the *data* key, with labels in the *labels* key. Each dynspec is treated as a single-channel image (intensity over time–frequency, so the pipeline forces the tensor shape to $(N, H, W, 1)$ and then converts it to PyTorch’s channel-first format $(1, H, W)$ per sample, where N is the total number of labeled images (e.g., 6555), H is height of image/dynspec matrix (e.g., 800), W is width of the image/dynspec matrix (e.g., 500) and converted all of the data into single-channel grayscale format.

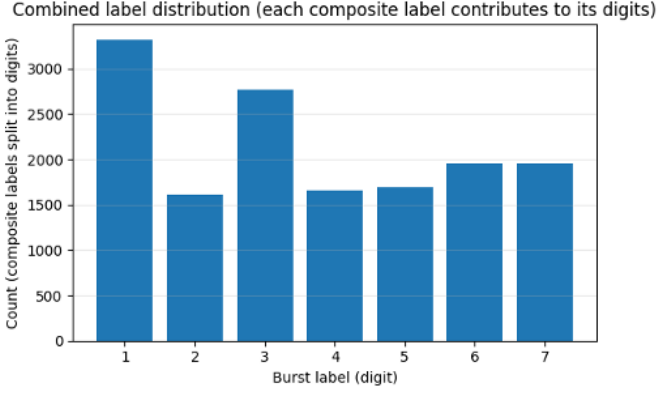


Fig. 13: Distribution of classes after synthetic augmentation [IV-C]

Because the labels can be composite (e.g., "24" meaning two bursts in one observation window), the code first converts each raw label into a list of digits via `parse_label_string()` function, then maps it into a multi-hot vector $Y \in \{0,1\}^C$ using `to_subset_multihot()` function. This turns "multi-label classification" into "7 independent binary decisions" for each image. For example, an image which contains a Type 2 and a Type 4 burst would be converted into label format $\{0,1,0,1,0,0,0\}$. This is called as 'multi-hot encoding'.

2) Train/val/test splitting and leakage control:

The data split parameters have been kept according to the standard machine learning conventions, with `TrainData/ValData/TestData` splitting 6555 real (non-synthetic) data into so-called stratified splits of 70%/15%/15%. Note that the generated synthetic data is only added to the training dataset after the splitting procedure. The summary of data splits for this model is as follows:

- TrainData = 70% of 6555 real + 1500 syn per T2,T4,T5
- ValidationData = 15% of 6555 real
- TestData = 15% of 6555 real

After adding synthetic data samples into the labeled dataset used here, a better-balanced distributions of SRB labels was achieved, as can be seen in Fig. 13. Two important leakage controls are embedded in the workflow:

- Persistent splits (`split_path`), the same indices are reused across every run of this baseline model, so improvements reported for Model 2 are attributable to changes in the model or the synthetic-augmentation configuration, not to a different train/val/test partition. This split (`splits_yolo_labelled7_filtered_seed42.npz`) is generated once from the class-filtered labeled pool described in Section IV-D, using a stratified 70/15/15 partition with a fixed seed (42), and is reused unchanged across Experiments 1–4 in Fig. 4. Note that this split occupies a different index space from the unfiltered-anchored split used for the self-supervised model (Appendix XI): the two use matched ratios and the same seed, but are not

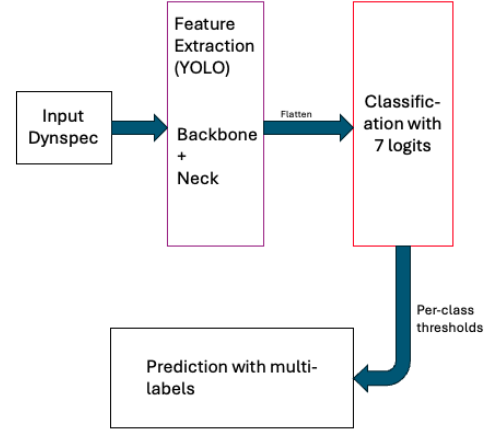


Fig. 14: Flow diagram representing Supervised Multi-label Classification IV-D

guaranteed to select the identical underlying observations. This is a limitation discussed in Section VI.

- cDCGAN data appended to TRAIN only: any synthetic HDF5 files are loaded, filtered with the same label rules, converted to multi-hot, and concatenated only into `X_train, Y_train`. Validation and test remain purely real images, which keeps reported performance honest ("since, this work dont want to test on the own synthetic generated samples"). This was a major gap in Zhang et al. [?], research. Refer to the section VI. Splitting the real dataset first by keeping test and validation set separately before adding synthetic data to remaining training dataset mirrors good machine learning practice which lacked in earlier SRB work where synthetic data is used for training and validation augmentation. Doing this ensures robust evaluation on real observations [?].

3) *Network Architecture*: This model borrows the feature extraction philosophy of "You Only Look Once" **YOLO**-family networks, but does not perform detection (no anchors, no bounding boxes, no grid prediction). Instead, it turns the YOLO backbone+neck into a single image embedding and attaches a multi-label classification head. You can think of it as:

Dynspec $\rightarrow$ Backbone (feature extraction) $\rightarrow$ Neck (context) $\rightarrow$ Flatten + MLP (classification) $\rightarrow$ 7C logits $\rightarrow$ sigmoid $\rightarrow$ 7 independent probabilities.

This is specifically attractive for LOFAR dynspecs because this work often care about "does type-k exist anywhere in this image?" rather than "where is it?", which avoids the manual bounding-box labeling burden that multi-detection pipelines typically require [?].

a) Backbone (Feature extraction) **YOLOv8 adapted**:

The backbone is a sequence of convolutional stages that progressively downsample the input while increasing channel capacity. The adapted version here reduces convolutional and C2F layers by half to decrease training time. The network

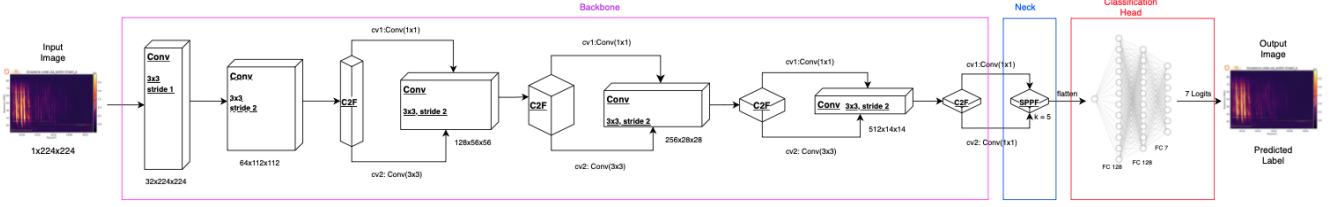


Fig. 15: Network Architecture of Supervised Multi-label Classifier described in Sec. IV-D. Backbone is YOLOv8-inspired feature extractor with a SPPF neck. The classification head outputs predicted logit scores which are converted into multi-label predictions using per-class thresholds.

architecture is as follow:

- ConvBlock = Conv2d $\rightarrow$ BatchNorm $\rightarrow$ SiLU.
 - It learns local patterns (edges, streaks, bands), normalizes activations for stable training, and uses SiLU for smooth gradients (often helpful when signals are faint and this work don't want "dead" activations).
 - Why it works for dynspecs: many SRB types are defined by oriented, elongated structures (drift lanes, broadband patches), which are naturally captured by stacked conv kernels.
- Downsampling occurs with stride-2 conv blocks:
 - Input 224 $\times$ 224 becomes 112 $\times$ 112, then 56 $\times$ 56, then 28 $\times$ 28, then 14 $\times$ 14.
 - This matters because burst morphology often spans large time–frequency extents; deeper layers need a large receptive field to classify correctly.
- C2F module:
 - It splits the incoming tensor into two transformed branches (cv1, cv2), runs one branch through a small stack of 3 $\times$ 3 convs, then concatenates and fuses with a 1 $\times$ 1 conv.
 - What it does conceptually: encourages feature reuse (some channels keep "clean" low-level patterns while others get progressively refined), and gives extra capacity without exploding compute.

Net effect: the backbone outputs a dense feature map of shape **(512, 14, 14)** per image that encodes both fine texture and coarse structure of a the dynspec.

b) Neck (context aggregation) "SPPF": After the backbone, this work use SPPF (Spatial Pyramid Pooling – Fast):

- It first reduces channels (1 $\times$ 1), then applies max-pooling multiple times with stride 1 (effectively simulating multiple receptive-field sizes), and concatenates pooled outputs.
- What it does: injects multi-scale context without changing spatial resolution. For SRBs, this helps because a "Type 2-like" structure might be thin and diagonal in one observation, but broader or fragmented in another; pooling at multiple effective scales helps stabilize the representation.

In classic detection with YOLO, this neck would feed the images to a bounding boxes detection head. Here, it feeds

a multi-label classifier, but the benefit remains same from YOLO: richer context before the final decision.

c) Classification Head: MLP $\rightarrow$ logits $\rightarrow$ prediction:

This is where the backbone feeds the image data after feature extraction via neck to perform a classification task. The Multi-layered perceptrons (MLP) are Fully-Connected-Neural-Network (FCNN) with layers of neurons connected one-to-all to produce 7 output scores (logits) at the final layer.

The FC layers are structured as follows:

- 1) Flattens the entire neck output (512 $\times$ 14 $\times$ 14 $\rightarrow$ 100k features).
- 2) Passes it through a 2-layer MLP with dropout:
 - fc1 (flat $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5)
 - number of neurons: 128
 - fc2 (128 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5)
 - number of neurons: 128
 - fc3 (128 $\rightarrow$ C) producing logits.
 - C = 7 $\rightarrow$ neurons: 7 logits, where 7C = num_classes

What logits mean in multi-label: each logit is an unbounded score for "presence of class C". They are not constrained to sum to 1 (unlike softmax), which is exactly what this work want when multiple bursts can co-occur.

4) Training objective: Because labels are multi-hot, the loss is computed independently per class, then averaged. This work use Multilabel FOCAL loss for this.

$$FL(z_c, y_c) = -\alpha_{t,c} (1 - p_{t,c})^\gamma \log(p_{t,c}) \quad (9)$$

over C=7 classes (per sample) is usually averaged:

$$\mathcal{L}_{FOCAL} = \frac{1}{c} \sum_{c=1}^c FL(z_c, y_c) \quad (10)$$

The focal loss wraps BCE (eq. 4) and downweights "easy" decisions:

- If an example/class image-label pair is already predicted correctly with high confidence, its contribution shrinks.
- This prevents training from being dominated by abundant easy negatives (especially relevant when the "Nothing (6)" class or negatives for rare bursts overwhelm gradients).

This is particularly important for dynspecs where many images contain weak or ambiguous signatures - focal loss keeps the

model spending gradient budget on the hard borderline cases rather than polishing already-solved negatives. In physical terms, focal loss prioritizes ambiguous or weak burst signatures, which correspond to physically marginal events near the detection threshold rather than obvious strong emissions.

5) *Optimization and Stability*: The training loop is a standard supervised loop with several training stabilization features like:

- AdamW optimizer: adaptive learning + decoupled weight decay (helps regularization when the head is dense and high-capacity) [?].
- Gradient clipping (max_grad_norm=5.0): prevents occasional exploding gradients (common when some batches contain unusual intensity artifacts).
- ReduceLROnPlateau on validation loss: if val loss stops improving, Learning Rate (LR) is halved down. This typically produces a “staircase” LR schedule that lets us keep learning without oscillating.
- Early stopping monitors validation loss: If the validation loss increases for more than 15 epochs, the training would stop preventing the model from overfitting to the training dataset.

6) *Output: Multi-label prediction using dynamic per-class thresholds*: At inference, logits are converted to probabilities via sigmoid [3].

Then to pick classification thresholds to turn probabilities into binary predictions:

- During training/validation, a single scalar threshold is used (default 0.5) to compute running F1 scores.
- After training, the pipeline learns **per-class thresholds** of predicted label from the validation set:
 - It computes a precision–recall curve per class and selects the threshold that maximizes F1.
 - Thresholds are clipped into [0.05, 0.95] to avoid degenerate “always on/off” behavior.
 - Example, an image is predicted to have a type 2 burst if the predicted probability by the model is more than the calculated per-class decision threshold calculated from the validation loop.

This is a big deal in SRB classification since different burst types often have systematically different score calibration depending upon their morphology. Having a single decision threshold like 0.5 for all different types of bursts is not ideal, since this work can miss rare classes and multi-label prediction can collapse. So per-class thresholds usually improve recall without changing the network at all.

7) *Reporting and Diagnostics*: After the model training is done, this work use 15% of the Unseen Test Data for measuring performance using metrics such as:

- 1) Overall loss, micro/macro F1, subset accuracy.
 - 2) Per-class precision/recall/F1 + ROC-AUC and PR-AUC.
- ‘Support’ is the number of positive test samples for a class. ‘ROC–AUC’ measures how well the model separates positives from negatives across thresholds, while ‘PR–AUC’ summarizes the precision–recall trade-off and is typically more in-

TABLE III: Supervised model trained on filtered data (no synthetic) and evaluated on filtered test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.6896	0.7778	0.8861	0.6250	0.2449	0.8146	0.9698
Recall	0.9927	0.4375	0.9272	0.2083	0.4000	0.9864	0.7679
F1	0.8138	0.5600	0.9061	0.3125	0.3038	0.8923	0.8571
ROC-AUC	0.9115	0.9531	0.9846	0.8814	0.9329	0.9672	0.9624
PR-AUC	0.6900	0.6758	0.9660	0.4572	0.2327	0.8785	0.9028
Support	273	16	302	24	30	294	293

TABLE IV: Supervised model where test condition is kept similar to Zhang *et al.* [?], where the test split contains both real LOFAR samples and synthetic (cDCGAN) samples generated from the same overall dataset split used in training (see Section VI). Metrics are reported per class.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.7817	0.9872	0.8642	0.9837	0.9956	0.7912	0.9696
Recall	0.9779	0.9627	0.9663	0.9719	0.8824	0.9796	0.7611
F1	0.8689	0.9748	0.9124	0.9778	0.9356	0.8754	0.8528
ROC-AUC	0.9680	0.9954	0.9773	0.9949	0.9866	0.9799	0.9769
PR-AUC	0.8922	0.9829	0.8990	0.9867	0.9361	0.8594	0.8929
Support	498	241	415	249	255	294	293

formative for rare, imbalanced classes. The detailed per-class performance metrics across different experiments are reported in figures III, IV, V, and VI.

TABLE V: Supervised model trained on filtered data with synthetic augmentation and evaluated on filtered test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.6510	0.5000	0.8284	0.5455	0.2308	0.7920	0.9770
Recall	0.9634	0.3750	0.9272	0.5000	0.1000	0.9456	0.7235
F1	0.7770	0.4286	0.8750	0.5217	0.1395	0.8620	0.8314
ROC-AUC	0.9103	0.9088	0.9735	0.9138	0.8658	0.9543	0.9427
PR-AUC	0.6973	0.4841	0.9227	0.5160	0.1710	0.8449	0.8813
Support	273	16	302	24	30	294	293

TABLE VI: Supervised model trained on mixed (filtered+unfiltered) data with synthetic augmentation and evaluated on unfiltered (raw) test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.4224	0.5000	0.4840	0.0351	0.0460	0.6440	0.2871
Recall	0.9377	0.5625	0.9007	0.7917	0.7667	0.9354	0.9181
F1	0.5825	0.5294	0.6296	0.0673	0.0868	0.7628	0.4374
ROC-AUC	0.7383	0.9126	0.7417	0.6299	0.6171	0.8525	0.4825
PR-AUC	0.4165	0.5456	0.4537	0.0386	0.0545	0.6605	0.3191
Support	273	16	302	24	30	294	293

XI. APPENDIX: SELF-SUPERVISED CLASSIFICATION MODEL IN DETAIL

This section fully documents the Self-Supervised Classification Model used (Section refsec: SSL).

A. Multi-source SSL Pre-training

1) *Pretraining Datasets*: The pretraining dataset is implemented in *MultiSSLDataset* class, and it builds one combined pool of SSL pretraining samples from three sources:

- 1) Labeled twins (70% of labeled dataset): The labeled dataset was split during supervised learning and stored for each index as `train_ids` (70% of 6555 labeled images), `val_ids` (15% of 6555) and `test_ids` (15% of 6555) in the memory. The `train_ids` was loaded from `splits_npz` function as a dataset used for pretraining. The dataset loads two base arrays: one from the filtered labeled HDF5 and one from the unfiltered labeled HDF5. These are returned as a paired “twin” sample (`kind='twin'`). In this phase all labels are dropped and inputs everything as sets of unlabeled images. Importantly, the code asserts that filtered and unfiltered labeled datasets have the same length and uses identical indices, ensuring that each twin is the same observation under two preprocessing regimes.
- 2) Unlabeled filtered dynspecs: Every sample from the unlabeled filtered HDF5 (`total_unl = 43107` images) is added as its own SSL item (`kind='ufil'`). These do not have a twin view from an unfiltered counterpart, so SSL generates two views from the same base image via augmentation.
- 3) Synthetic dynspecs (also treated as unlabeled): Each synthetic HDF5 contributes its full set of samples (`kind='syn'`). Even though these files contain labels as well, SSL pretraining ignores them by design; synthetic samples are simply extra draws from a plausible burst-like distribution that enrich the pretraining pool.

The split integrity is enforced right at startup: the script loads `train_ids`, `val_ids`, and `test_ids` splits. Only `train_ids` are used to form twin entries, which prevents any representation pretraining signal from leaking into evaluation splits. All the data sourced (except labeled-twins) then goes through an augmentation process to create “positive-pairs” of same image.

2) *Input normalization and Augmentations*: Before any augmentation is done to create “positive-pairs”, each dynspec is normalized using `robust_minmax` function, which rescales intensities using the 1st and 99th percentiles (instead of min/max). This is a practical choice for radio spectra where a few extreme pixels (RFI spikes, bright bursts) can dominate min/max scaling and collapse dynamic range. The normalized image is resized to a fixed spatial size of 224×224. The augmentation pipeline is intentionally “dynspec-safe” using *DiffAug* [?]: it injects variability while preserving the kind of structure that defines burst morphology.

- Jitter: This simulates gain variation and measurement noise without altering semantic structure.

- Mild warp: small rescale-and-resample to mimic slight distortions while keeping overall geometry realistic.
- SpecAug-style masks: random time masking (vertical bands) and frequency masking (horizontal bands). These force the encoder to rely on global context and partial evidence, which is useful for multi-label events where some components may be faint or partially occluded.

These augmentations are performed on unlabelled and synthetic datasets only. For labeled twin samples specifically, the two branches do not start from the same base image: one branch begins from the filtered observation and the other from the unfiltered one. Both does not receive independent augmentations. This means the “positive pair” is defined at the level of the underlying event between filtered and unfiltered image making the encoder invariant to RFI noises.

3) *Encoder and Projection head*: This subsection describes the representation engine of the SSL pre-training method: a convolutional neural-network $f_{\theta}(\cdot)$ that converts an input dynspec into a compact feature representation, and a projection head $g_{\phi}(\cdot)$ that maps those features into an embedding space where contrastive learning is applied. Refer equation [14]. The core goal here is not feature extraction, but learning a feature space in which two different “views” of the same underlying SRB image are close, while views of different observations are far apart. In this pipeline, the encoder is the reusable asset: it learns morphology- and texture-aware dynspec features that later gives the input to the supervised multi-label classifier head. Figures [5] and [6] demonstrates the flow for this training processes.

Architecturally, the encoder is a CNN tailored to dynspec textures and multi-scale structure as shown in Fig. [16]. It follows a staged design with increasing channel depth, which matches the intuition that earlier layers learn local primitives (edges, thin streak segments, small blobs), while deeper layers learn larger motifs (broadband emission, drift structures spanning time, patchy backgrounds). The *stem* expands the single-channel dynspec into 32 channels, creating a richer basis for feature extraction. The downsampling stages increase depth from 32→64→128 while reducing spatial resolution. Instead of strided convolutions alone, this work use BlurPool downsampling, which is better for dynspecs because many meaningful patterns are fine-grained and aggressive downsampling can alias these cues into misleading textures. BlurPool acts as an anti-alias filter before subsampling, making the learned representation more stable to resizing and to small spatial perturbations introduced by augmentation.

After downsampling, the *mid* stage (128→256) increases representational capacity while maintaining locality; this stage is where the encoder can begin combining local streak fragments into more structured “proto-features” (e.g., diagonal ridges, curved fragments, or repeated fine textures). The *tail* stage (256→512) produces the highest-level feature maps. Dropout feature is also implemented here since it helps to reduce overfitting (especially helpful when training on mixed sources like unlabeled real + synthetic). SPPF blocks aggregates information across multiple receptive-field scales. This

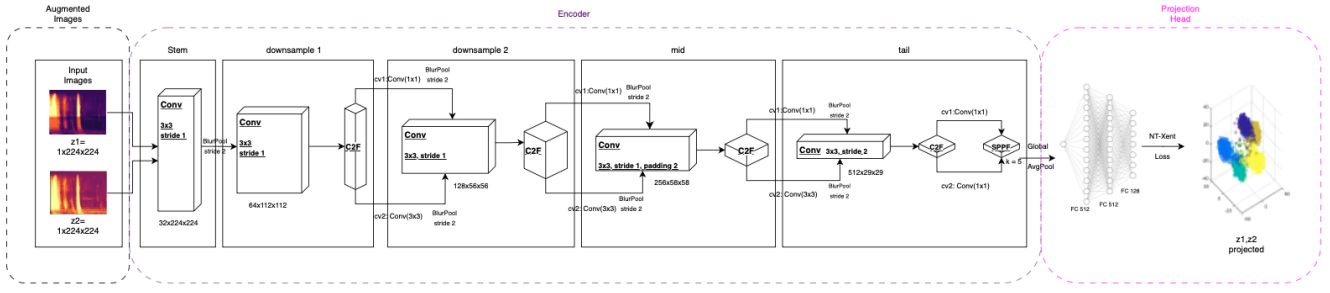


Fig. 16: Network Architecture for the SSL pre-training XI-A. Fine-tuning architecture remains same as supervised multi-label classifier (in figure 6, replaced feature extractor for pre-trained encoder)

is particularly aligned with dynspec data: this work wants the model to simultaneously “notice” local thin features (Type III-like fine structure) and large-scale regions (broad continuum like Type IV). SPPF blocks resize images at multiple effective scales and fusing them back into a unified representation.

The C2F blocks are lightweight YOLOv8 feature mixing modules inserted throughout the network. Functionally, they increase depth and nonlinearity without the compute cost of a heavy residual stack. By splitting channels into two branches, transforming one branch through a small conv stack, and then concatenating and fusing, C2F encourages the encoder to learn both (i) a relatively direct pathway that preserves essential signal and (ii) a transformed pathway that captures more abstract combinations. For dynspecs, this is beneficial because the representation needs to preserve faint but meaningful structures while also learning higher-order texture groupings that differentiate event morphologies.

On top of the encoder, pretraining adds a projection head because contrastive learning works best when the loss is applied in a space designed for metric separation rather than directly on the raw encoder features [?]. In this implementation, the encoder output feature map is first compressed using global average pooling, producing a 512-Dimensional vector. This vector is passed through an MLP $512 \rightarrow 512 \rightarrow 128$, and then L2-normalized, yielding a unit-norm embedding. The key conceptual point is: the projection head is where the contrastive objective exerts its strongest geometric pressure. That pressure shapes the projection space to be highly discriminative at the instance level, while the encoder is encouraged to learn transferable features that remain useful when this work later discards the projection head and attaches a classification head. This separation is one reason why SimCLR-style pretraining tends to transfer well: it prevents the encoder from being overly specialized to the exact contrastive similarity geometry and instead promotes general-purpose features that a classification head can reuse [?].

4) *Contrastive loss (NT-Xent)*: Training minimizes the **NT-Xent** contrastive objective [?] with temperature $\tau = 0.2$. For a minibatch of size B , two stochastic augmentations are sampled per input dynspec, producing $2B$ projected embeddings

$\{z_k\}_{k=1}^{2B}$ (typically ℓ_2 -normalized). For an anchor embedding z_i , the corresponding embedding from the other view of the same underlying event is treated as the positive $z_{p(i)}$, while all remaining embeddings in the batch act as negatives.

We define cosine similarity as

$$\text{sim}(z_i, z_k) = \frac{z_i^\top z_k}{\|z_i\|_2 \|z_k\|_2}, \quad (11)$$

and the per-anchor NT-Xent loss as

$$\ell_{i,p(i)} = -\log \frac{\exp(\text{sim}(z_i, z_{p(i)})/\tau)}{\sum_{k=1}^{2B} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}. \quad (12)$$

The batch objective averages over all $2B$ anchors:

$$\mathcal{L}_{\text{NT-Xent}} = \frac{1}{2B} \sum_{i=1}^{2B} \ell_{i,p(i)}. \quad (13)$$

Expected loss behavior: $\mathcal{L}_{\text{NT-Xent}}$ typically decreases as seen in figure [17] as the encoder learns augmentation-invariant representations that consistently align positive pairs while separating negatives. Unlike supervised training, the contrastive loss does not map directly to per-epoch classification accuracy; the primary artifact is the saved encoder weights checkpoint (e.g., `encoder_ssl.pth`), which captures the learned representation used for fine-tuning.

5) *Training Process*: During SSL pre-training, the encoder is not learning to predict burst types. Instead, it is learning a mapping $f_\theta(\cdot)$ (the encoder parameters θ) such that two views of the same underlying dynspec event “positive-pair” land close in representation space, and views from different events “negative-pair” land far apart. The optimization signal comes from the contrastive loss (NT-Xent), but importantly that loss is computed on the projection vectors produced by the projection head $g_\phi(\cdot)$. A systematic flow of different steps are represented in Fig. [5 and 6].

(dynspec) $\rightarrow$ Encoder $\rightarrow$ feature map $\rightarrow$ flatten $\rightarrow$ projection MLP $\rightarrow$ normalized embedding $\rightarrow$ contrastive loss

Even though the loss is computed after the projection head, the gradient flows backward through the head and into the encoder. So, the encoder weights are updated to make the

post-encoder representations more “view-invariant” and more “instance-discriminative”. Concretely, in each minibatch this work build two tensors x_1 and x_2 containing two randomized views of each sample. Then the positive-pair embeddings are computed:

$$z_1 = g_\phi(\theta(x_1)), \quad z_2 = g_\phi(\theta(x_2)) \quad (14)$$

These are the pair embeddings. The Nt-Xent loss from equation [13] then compares similarities computed from equation [11] between these embeddings. If the encoder produces features that change drastically under jitter/warp/masking (or under filtered vs unfiltered twins), the similarity between z_1 and z_2 for the same event drops, and the loss increases. Back-propagation responds by adjusting convolution filters, which results in deeper feature compositions capture so that the embedding becomes stable under exactly those perturbations.

That means the SSL pre-training objective is implicitly training the encoder to:

- 1) Ignore nuisance factors such as:
 - intensity scaling (gamma/brightness/contrast),
 - small time/frequency masking,
 - small geometric warps,
 - and crucially for the setup: the filtered vs unfiltered domain shift (because twins are paired across preprocessing).
 - This is the “invariance” to noise power.
- 2) Keep event identity information: While becoming invariant, the encoder also keeps enough information to distinguish different samples within a batch. That pressure prevents collapse into a trivial constant embedding and encourages learning of various meaningful structures of SRBs (e.g., drift lines, broadband patches, texture statistics), because those are exactly the cues that remain consistent across augmentations.
- 3) Learn multi-scale morphology: Downsampling stages plus SPPF in the encoder can represent both local patterns (thin drift tracks) and global context (broadband structure and large-scale background). The contrastive loss “rewards” features that consistently reappear across both views, which tends to be morphology rather than pixel noise.

The encoder weights are optimized with:

- AdamW: updates weights using adaptive gradients while applying decoupled weight decay. Weight decay encourages simpler solutions and reduces overfitting to augmentation quirks.
- Cosine Learning Rate (LR) with warmup: early steps ramp up LR to avoid unstable updates when embeddings are random, then gradually reduce LR to refine invariances.
- AMP + GradScaler: mixed precision speeds training and stabilizes gradients.
- Gradient clipping: prevents occasional hard-negative batches from causing exploding updates.

So from a “what it does” perspective: the optimizer is repeatedly nudging the encoder to produce a representation where “same event under perturbations” is a tight cluster, and “different events” are well separated — using a training pool that includes real unlabeled data and synthetic samples, while explicitly teaching invariance across filtered/unfiltered twins.

B. Multi-Label Classification SSL Fine-Tuning

Once SSL pre-training finishes, the encoder weights θ already encode a strong prior. They can now extract features that are robust to the variations that happen naturally in LOFAR dynspecs. Fine-tuning then re-purposes those features for multi-label decision making by learning a supervised classifier using these feature to class logits.

1) *Input Data protocol*: Fine-tuning is built around a strict and reproducible data protocol:

- Splits are anchored on the unfiltered labeled dataset via a splits_npz file containing keys train_ids, val_ids, and test_ids, generated once (stratified 70/15/15, seed=42) directly over the full unfiltered labeled index space. This split file is independent of the one used for the baseline supervised model (Model 2, Appendix X) and is shared, unmodified, between SSL pretraining (Appendix XI-A) and this fine-tuning stage. IV-D
- SSL fine-tuning training uses filtered images from the train_ids indices ($X_{fil}[\text{train_ids}]$). This leverages the cleaner representation for gradient updates for supervised training of SSL fine-tuning phase.
- Validation and test use unfiltered images only ($X_{unf}[\text{val_ids}]$, $X_{unf}[\text{test_ids}]$). This makes evaluation intentionally harder and directly measures whether the pipeline learned semantics beyond preprocessing artifacts.

This is also consistent with the SSL pretraining stage: SSL pretraining and fine-tuning both draw exclusively on the train_ids partition of this splits_npz file, and neither stage ever observes val_ids or test_ids. The result is evaluation done on data that is purely unseen by Model 3 throughout pretraining and fine-tuning. As noted in Appendix X, this held-out set is independently generated from, and therefore not sample-identical to, the held-out set used to evaluate Model 2.

Optionally, synthetic-generated HDF5s are appended only to the training set as labeled samples. The code loads their labels, converts them to the same multi-hot label space, filters, and concatenates them with the real filtered training data. Val/test datasets remain purely real unfiltered “raw” LOFAR data.

2) *Feature Extractor (encoder) and Classification head*:

The fine-tuned classifier reuses the exact same encoder from SSL pretraining. The encoder is loaded back as a feature extractor using *encoder_ssl.pth* XI-A4. The multi-label classification architecture here is similar to supervised (model 2) except for encoder as:

- 1) Encoder output $\rightarrow (B, 512, h, w)$
- 2) Global average pooling $\rightarrow (B, 512)$
- 3) Fully-connected head: $512 \rightarrow 256 \rightarrow 128 \rightarrow 7C$ (logits)

The model outputs logits per class. Multi-label probabilities are produced with sigmoid independently for all 7 classes.

3) *Training and Loss Function*: Fine-tuning follows the same outer training loop structure as the supervised model (mini-batches, forward pass, loss, backpropagation, optimizer step, validation), but the key difference is what is being trained and why. Instead of learning the entire feature extractor from scratch, the model starts with a pretrained SSL encoder (loaded from `encoder_ssl.pth`) that already encodes dynspec morphology, textures, and multi-scale structure in a way that is robust to augmentation, filtering differences, and unlabeled-domain variability. In short, the feature extractor here already knows what the burst structure look like and "it already knows what to look for in the input image" as it is explicitly trained to do so in the pre-training process.

In the training loop, the forward pass is:

- 1) Input dynspec $\rightarrow$ encoder: a training sample is passed through the encoder to produce deep feature maps (B,512,h,w).
- 2) Global pooling $\rightarrow$ feature vector: GlobalAvgPool compresses the spatial map into a 512-dimensional descriptor per sample. This step is better than normal flattening.
- 3) Classification head $\rightarrow$ logits: the pooled descriptor goes through an MLP head (512 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 7C) to output logits (one logit per class).

Although the loss function is standard multi-label BCE loss (4), the important part is how the weights are actually optimized through feedforward + backpropagation. For each batch:

- The model produces 7 logits, i.e. 7 predicted output scores for each input image
- BCE Loss compares predicted label against the multi-hot ground truth $y \in \{0,1\}^C$ and produces a scalar loss L .
- Backpropagation computes gradients $\nabla_{\theta} L$ and $\nabla_{\phi} L$ through the classification head and (when unfrozen) the encoder, telling each parameter how it should change to reduce the loss.
- AdamW then updates parameters using adaptive moment estimates (first and second moments of gradients) plus decoupled weight decay. Practically, AdamW is a good fit here because fine-tuning is sensitive: gradients can be noisy (multi-label imbalance, heavy augmentation), and AdamW tends to converge faster and more stably than vanilla Stochastic Gradient Decent in this regime.

A critical stability mechanism in the fine-tuning is staged training (warmup + progressive unfreezing), which controls how much the encoder is allowed to change:

- Head warmup (encoder frozen): for the first 5 epochs, all encoder parameters are frozen. Only the classification head learns. This makes the head quickly adapt to the label distribution without immediately overwriting the SSL encoder representation (catastrophic forgetting risk is highest at the beginning).
- Progressive unfreezing: after warmup, this work gradually unfreeze deeper encoder blocks (tail first, then mid, then

earlier stages). When this work unfreeze, the optimizer is rebuilt with two parameter groups:

- head at learning rate lr
- encoder at a smaller learning rate $lr * lrd$ (e.g., $lrd=0.1$)
- This is the fine-tuning "sweet spot": the head learns aggressively, while the encoder adapts slowly to the supervised target.
- Step-level LR scheduling: each optimization step applies a cosine schedule with warmup (cosine_with_warmup function), which helps avoid unstable early updates and then gradually anneals learning rates as training progresses.
- AMP + gradient clipping: Automatic Mixed Precision (AMP) reduces memory/boosts throughput; gradient clipping prevents rare unstable batches from producing extreme gradient updates.

Validation loss is computed every epoch on the VAL split (which in the strict setup is real unfiltered data only, not seen in any training stage). This validation loss acts as a generalization check: even if training loss decreases, validation loss will flatten or increase if the model starts overfitting to filtered training appearance or to synthetic artifacts as seen in figure [17]. The training loop is halted if the validation loss starts increasing after 10 epochs to prevent overfitting. In other words, the train/val loss gap is an early indicator of whether the model is learning transferable dynspec cues versus memorizing training-specific textures. this work additionally stabilize validation evaluation by using Exponential Moving Average (EMA) weights: during training an exponential moving average of parameters is maintained; for validation this work temporarily apply EMA weights, compute metrics/loss, and then restore the raw weights. This typically reduces noise in validation curves and gives a better estimate of the model that generalizes.

4) *Output and Per-class-thresholds*: At inference, logits are converted to probabilities via sigmoid 3. This yields a probability vector $p \in [0,1]^C$ for each test sample. Converting probabilities into a multi-label decision requires a threshold rule. This pipeline uses a per-class threshold calibration learned on the validation set (same as supervised (SL) model):

- For each class C , this work search over candidate thresholds and pick the one that maximizes F1 on VAL (per_class_thresholds).
- These thresholds are then frozen and applied to test set, producing multiple binary predictions for single input image.

This is important because a single global threshold (like 0.5) is rarely optimal under multi-label imbalance; per-class calibration lets rare classes use lower thresholds if needed while keeping frequent classes from over-firing.

5) *Reporting and Diagnostics*: On the test set, evaluation mirrors the supervised (SL) pipeline so results are directly comparable. this work use 15% of the Unseen Test Data from `test_ids` for measuring performance using metrics such as:

- 1) Overall loss, micro/macro F1, subset accuracy.

TABLE VII: Self-supervised (pretrained encoder) without synthetic data; evaluated on filtered test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.7578	0.6111	0.9244	0.7500	0.3960	0.8579	0.9929
Recall	0.8272	0.8462	0.8918	0.6923	0.7692	0.9691	0.9521
F1	0.7910	0.7097	0.9078	0.7200	0.5229	0.9101	0.9720
ROC-AUC	0.9104	0.9963	0.9766	0.9601	0.9666	0.9944	0.9862
PR-AUC	0.8297	0.8068	0.9649	0.7681	0.6829	0.9689	0.9852
Support	382	13	425	26	52	162	292

TABLE VIII: Self-supervised (pretrained encoder) with synthetic data; evaluated on filtered test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.8047	0.9375	0.9296	0.7667	0.6176	0.9324	0.9873
Recall	0.8536	0.7895	0.9113	0.7667	0.5250	0.9857	0.9719
F1	0.8284	0.8571	0.9204	0.7667	0.5676	0.9583	0.9795
ROC-AUC	0.9448	0.9989	0.9856	0.9531	0.9700	0.9934	0.9966
PR-AUC	0.8904	0.9451	0.9798	0.8151	0.7043	0.9900	0.9912
Support	362	19	406	30	40	140	320

TABLE IX: Self-supervised (pretrained encoder) with synthetic data; evaluated on unfiltered (raw) test data.

Metric	Type 1	Type 2	Type 3	Type 4	Type 5	Nothing (6)	CasA (7)
Precision	0.7519	0.9412	0.8610	0.3571	0.5238	0.7861	0.9766
Recall	0.8167	0.6667	0.9298	0.6897	0.7674	0.9128	0.9733
F1	0.7829	0.7805	0.8941	0.4706	0.6226	0.8447	0.9750
ROC-AUC	0.9085	0.9926	0.9716	0.8995	0.9453	0.9785	0.9948
PR-AUC	0.8238	0.8424	0.9589	0.4964	0.5575	0.9094	0.9914
Support	371	24	413	29	43	149	300

2) Per-class precision/recall/F1 + ROC-AUC and PR-AUC. The detailed per-class performance metrics across different experiments are reported in figures VII, VIII, and IX.

XII. APPENDIX: SLURM JOB SUBMISSION AND HARDWARE CONFIGURATION

All experiments reported in this work were executed on the Spider High-Performance Computing (HPC) cluster operated by SURF (Netherlands). The cluster uses the SLURM workload manager for scheduling and resource allocation. General Simple Linux Utility for Resource Management (SLURM) usage and command syntax follow the official documentation provided at: <https://doc.spider.surfsara.nl/en/latest/index.html>.

Deep learning training and evaluation were performed on GPU compute nodes equipped with NVIDIA A100 GPUs with 40 GB memory. Jobs were submitted to the *gpu_a100_22c* or *gpu_a100_7c* partitions, corresponding to nodes with 22 or 7 CPU cores available per GPU, respectively. Each job requested a single A100-40GB GPU together with the allocated CPU cores for data loading and preprocessing. The NVIDIA A100 GPU provides high single-precision compute throughput and supports Tensor Core acceleration. The nominal power consumption of a single A100 GPU is up to 400 W, while the host CPU power envelope is typically in the range of 150–250 W per node. Training jobs were submitted using SLURM batch scripts (.sbatch files) with explicitly defined resource requirements. GPU access was requested using the `gres=gpu:1` directive, while CPU cores and memory were specified using `-cpus-per-task` and `-mem`. Job submission followed the standard SLURM batch workflow described in: https://doc.spider.surfsara.nl/en/latest/Pages/compute_on_spider.html.

A typical training job was launched using in the terminal:

```
sbatch train_model.sbatch
```

At runtime, SLURM allocated an exclusive compute node and exposed the assigned GPU via standard CUDA environment variables. Network training was executed inside an Apptainer (Singularity) container with GPU passthrough enabled, ensuring a consistent software environment across runs. SLURM job identifiers and log files recorded execution details such as resource allocation and runtime behavior. By re-submitting the same SLURM batch script to the *gpu_a100_22c* or *gpu_a100_7c* partition with identical resource specifications, all training experiments reported in this work can be reproduced on the Spider HPC cluster.

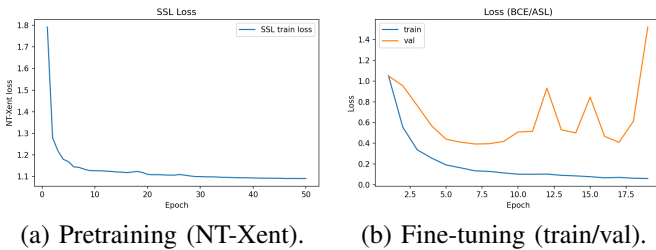


Fig. 17: SSL loss curves for the final experiment evaluated on the raw (unfiltered) test set. The fine-tuning was stopped automatically after a rise in validation loss to avoid overfitting.